

URBAN EYE : VISION-BASED AUTOMATED TRAFFIC OPTIMIZATION USING COMPUTER VISION

A Project Report

Submitted By

HIRVA TRIVEDI - 2303031050648

DARSHAN PATEL - 2303031050132

OGESH RUPARELIYA - 2303031050527

KARMDIPSINH VAGHELA - 2303031050666

in Partial Fulfilment For the Award of

the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Prof. Namrata Patel

Assistant Professor



VADODARA

November - 2026



PARUL UNIVERSITY

CERTIFICATE

This is to Certify that Project - II (303105423) of 7th Semester entitled “URBAN EYE : VISION-BASED AUTOMATED TRAFFIC OPTIMIZATION USING COMPUTER VISION” of Group No. CSE_151 has been successfully completed by

- Hirva Trivedi - 2303031050648
- Darshan Patel - 2303031050132
- Ogesh Rupareliya - 2303031050527
- Karmdipsinh Vaghela - 2303031050666

under my guidance in partial fulfillment of the Bachelor of Technology (B.Tech) in Computer Science and Engineering of Parul University in Academic Year 2026- 2027.

Date of Submission : _____

Prof. Namrata Patel

Project Guide

CSE, PIET

Parul University

Prof. Namrata Patel

Project Coordinator

CSE, PIET

Parul University

Dr. Shailendra K Mishra

Head of Department

CSE, PIET

Parul University

Acknowledgements

“The single greatest cause of happiness is gratitude.”

-Auliq-Ice

We would like to express our sincere gratitude to our project guide, **Prof. Namrata Patel**, for her constant guidance, valuable suggestions, and continuous support throughout the development of this project. Her expertise, encouragement, and constructive feedback enabled us to understand the concepts clearly and complete this project in an effective and systematic manner.

We are also thankful to the Head of the Department, **Dr. Shailendra K. Mishra**, for providing us with the opportunity, resources, and a supportive environment to successfully carry out this project.

We extend our sincere thanks to our project coordinator, **Prof. Namrata Patel**, for their valuable guidance, coordination, and encouragement throughout the various phases of this project.

We also express our heartfelt gratitude to the **Department of Computer Science and Engineering, Parul Institute of Engineering and Technology, Parul University**, for providing us with the necessary facilities and an excellent learning environment that made the successful completion of this project possible.

Finally, we sincerely thank our family members, friends, classmates, and everyone who directly or indirectly supported, motivated, and encouraged us throughout the development of this project.

Hirva Trivedi
Darshan Patel
Ogesh Rupareliya
Karmdipsinh Vaghela
CSE, PIET
Parul University,
Vadodara

Abstract

Traffic congestion has become a major challenge in urban areas due to the increasing number of vehicles and the limitations of conventional fixed-time traffic signal systems, which cannot adapt to real-time traffic conditions. This project, UrbanEye: Vision-Based Automated Traffic Optimization Using Computer Vision, aims to develop an intelligent traffic management system that dynamically optimizes traffic signal timings based on traffic density while providing priority to ambulances.

The proposed system processes traffic videos captured from CCTV cameras, with recorded videos used to simulate live traffic scenarios for testing and evaluation. Vehicle detection is performed using YOLOv8, while ByteTrack is employed for multi-object tracking to prevent duplicate vehicle counting. OpenCV is used for video frame processing and Region of Interest (ROI) selection, and detected vehicles are classified into different categories and assigned Passenger Car Unit (PCU) weights to calculate lane-wise traffic density.

In addition to optimizing individual junctions, the system coordinates multiple junctions along an ambulance's route through a green-corridor mechanism, clearing each signal in sequence just ahead of the vehicle's arrival so that delay is reduced across the entire route rather than at one intersection alone. A web-based dashboard, built using React and Chart.js, allows real-time monitoring of lane-wise vehicle counts, signal states, and traffic trends over time. When compared with conventional fixed-timer signals, the proposed system shows a noticeable reduction in average vehicle waiting time under varying traffic density.

Experimental evaluation demonstrates that the proposed system effectively estimates traffic density and dynamically allocates green signal durations while maintaining predefined minimum and maximum timing constraints. The system also successfully prioritizes ambulances by immediately assigning a green signal upon detection, thereby improving emergency response.

The proposed system is developed using Python, YOLOv8, OpenCV, and ByteTrack for vehicle detection and tracking. Flask and React are used for backend and frontend development, while Tailwind CSS and Chart.js provide the user interface and data visualization.

Keywords: Computer Vision, YOLOv8, ByteTrack, OpenCV, Vehicle Detection, Traffic Signal Optimization, Intelligent Traffic Management System

Table of Contents

Acknowledgements	iii
Abstract	iv
List of Tables	x
List of Figures	xi
1 Introduction	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Motivation	2
1.4 Aim	2
1.5 Objectives	2
1.6 Scope	3
1.7 Overview of Proposed System.....	3
1.8 Contributions of the Project	4
1.9 Organization of Report	4
2 Literature Survey	6
2.1 Review Methodology.....	6
2.2 Traditional and Optimization-Based Traffic Management Systems.....	6

2.3	Computer Vision and Deep Learning-Based Traffic Systems.....	7
2.4	IoT and Intelligent Transportation Systems.....	7
2.5	Summary of Literature Survey	7
2.6	Research Gaps	10
3	Analysis / Software Requirements Specification (SRS)	12
3.1	Existing System	12
3.1.1	Limitations of Existing System.....	12
3.2	Proposed System	13
3.3	Feasibility Study	13
3.3.1	Technical Feasibility	13
3.3.2	Economical Feasibility.....	14
3.3.3	Operational Feasibility	14
3.4	System Requirements	14
3.4.1	Functional Requirements	14
3.4.2	Non-Functional Requirements	14
3.4.3	Software Requirements	15
3.4.4	Hardware Requirements.....	15
3.4.5	Performance Requirements	16
3.5	Use Case	16
3.6	Constraints.....	16
3.7	Advantages of Proposed System.....	17
3.8	Summary	17
4	System Design	18
4.1	System Architecture.....	18
4.2	Module Design	19

4.2.1	Video Acquisition Module	19
4.2.2	Vehicle Detection Module	20
4.2.3	Vehicle Tracking Module	20
4.2.4	Traffic Density Estimation Module	21
4.2.5	Signal Timing Module	21
4.2.6	Helmet Detection Module	22
4.2.7	Dashboard Module	22
4.3	Data Flow Diagram.....	22
4.4	Overall System Workflow	24
4.5	User Interface Design	24
4.5.1	Landing Page	24
4.5.2	Home Dashboard	25
4.5.3	Monitor Dashboard	25
4.5.4	City Map Dashboard	28
4.5.5	Helmet Dashboard	29
4.5.6	Sustainability (SDG) Dashboard.....	29
4.5.7	Analytics Dashboard	30
4.6	Technology Stack.....	31
4.7	Design Considerations	32
4.7.1	Modularity	32
4.7.2	Scalability	32
4.7.3	Performance	32
4.7.4	Cost Effectiveness.....	32
4.7.5	Reliability.....	32
4.7.6	Maintainability	32
4.8	Summary	33

5	Methodology	34
5.1	Overview	34
5.2	System Workflow	34
5.3	Region of Interest (ROI) Selection	35
5.4	Vehicle Detection and Tracking.....	35
5.5	Traffic Density and Signal Timing.....	36
5.6	Emergency Vehicle Priority and Dashboard Visualization	38
5.7	Summary	39
6	Implementation	40
6.1	Overview	40
6.2	Backend Implementation	40
6.2.1	Video Processing.....	40
6.2.2	Region of Interest (ROI) Selection.....	41
6.2.3	Vehicle Detection	42
6.2.4	Vehicle Tracking	42
6.2.5	Traffic Density and Signal Timing.....	43
6.2.6	Emergency Vehicle Detection	43
6.3	Frontend Implementation.....	44
6.4	Experimental Results.....	44
6.4.1	Results Discussion	45
6.5	Summary	46
7	Conclusion	47
8	Future Work	48
8.1	Real-Time Implementation	48

8.2	Multi-Intersection Coordination	48
8.3	Advanced Intelligent Control.....	48
8.4	Improved Detection Robustness	49
8.5	IoT and Edge Deployment.....	49
9	Research Paper and Plagiarism Report	50
10	References	63

List of Tables

2.1	Summary of Literature Survey	8
4.1	Technology Stack Used in the Proposed System	31
5.1	System Workflow Steps	34
5.2	Vehicle Classes Detected	36
5.3	PCU Weights for Vehicle Types	36
5.4	Signal Timing Parameters.....	37
5.5	Dashboard Features	38
6.1	Experimental Results of the Proposed System	45
6.2	Performance Comparison of Proposed System.....	46

List of Figures

3.1	Use Case Diagram of Vision-Based Traffic Signal Optimization System	16
4.1	Overall System Architecture.....	19
4.2	Data Flow Diagram of the Proposed System	23
4.3	Landing Page of Urban Eye.....	25
4.4	UrbanEye Dashboard Before Video Input	25
4.5	Monitor dashboard of simulation mode	26
4.6	Information shown on dashboard.....	26
4.7	Monitor dashboard of Live mode.....	27
4.8	Information shown on dashboard.....	27
4.9	Additional Info shown on dashboard	28
4.10	City map dashboard	28
4.11	Helmet Detection Dashboard.....	29
4.12	Sustainability (SDG) Dashboard	30
4.13	Analytics Dashboard (Overview).....	30
4.14	Analytics Dashboard (Performance Analysis)	31
6.1	Traffic Video Processing	41
6.2	Region of Interest Selection.....	41
6.3	Vehicle Detection using YOLOv8	42
6.4	Vehicle Tracking using ByteTrack.....	42
6.5	Traffic Density and Adaptive Signal Timing	43
6.6	Emergency Vehicle Detection.....	44

Chapter 1

Introduction

1.1 Background

Although considerable progress has been achieved in intelligent transportation research, implementing adaptive traffic management systems in real-world environments remains a challenging task. Traffic conditions are highly dynamic and can change within a short period due to fluctuations in vehicle volume, accidents, weather conditions, pedestrian movement, or temporary road closures. As a result, a traffic control strategy must be capable of continuously observing the current situation and responding without delay. Systems that depend solely on historical traffic information often fail to accommodate these unexpected variations, leading to inefficient signal operation.

Video-based traffic analysis has emerged as a practical alternative because surveillance cameras are already installed at many urban intersections. Unlike conventional sensing technologies, camera-based solutions can capture comprehensive information about road activity, including vehicle count, traffic density, movement patterns, and lane occupancy. This rich visual information enables more informed traffic management decisions while reducing the need for additional infrastructure. Furthermore, software-based solutions can be upgraded as computer vision algorithms continue to improve, making them more adaptable than hardware-dependent systems.

The effectiveness of a vision-based traffic management system largely depends on the accuracy of vehicle detection and tracking. Reliable detection ensures that vehicles of different categories can be identified under varying lighting and traffic conditions, whereas robust tracking prevents repeated counting of the same vehicle as it moves through consecutive video frames. Together, these capabilities provide a consistent representation of traffic flow, which is essential for estimating congestion levels and supporting adaptive signal timing.

1.2 Problem Statement

Conventional fixed-time traffic signal systems are unable to respond to continuously changing traffic conditions. Since equal signal duration is allocated irrespective of vehicle density, heavily

congested lanes often experience long waiting times while lanes with little or no traffic continue receiving unnecessary green signal time. This leads to traffic congestion, inefficient road utilization, increased fuel consumption, and higher emissions.

Therefore, there is a need for an intelligent traffic signal management system capable of monitoring traffic conditions using computer vision and dynamically adjusting signal timings according to real-time traffic density.

1.3 Motivation

The motivation for this project comes from the gap between the traffic information available through existing surveillance infrastructure and the limited use of that information for intersection-level decision making. CCTV cameras continuously capture visual information about road conditions, yet converting these observations into useful control decisions requires an automated mechanism capable of interpreting the scene and responding accordingly. This creates an opportunity to explore how computer vision can transform ordinary traffic video into actionable information for signal management.

Another motivation is to develop a system that brings several traffic-management functions together rather than treating vehicle detection, traffic measurement, and signal control as separate tasks. The project therefore focuses on building an integrated prototype in which information obtained from traffic video can directly influence signal decisions, while emergency situations can be handled separately when required.

1.4 Aim

The aim of this project is to develop a vision-based intelligent traffic signal management system that automatically analyzes traffic density, dynamically optimizes traffic signal timings and prioritize emergency vehicles using computer vision and deep learning techniques.

1.5 Objectives

The main objectives of the proposed system are:

- To detect and classify vehicles using the YOLOv8 object detection model.
- To track detected vehicles using the ByteTrack algorithm to prevent duplicate counting.
- To calculate lane-wise traffic density using weighted vehicle classification.

- To dynamically allocate green signal duration based on calculated traffic density.
- To maintain minimum and maximum green signal limits for fair traffic distribution.
- To provide priority to emergency vehicles such as ambulances.
- To develop an interactive dashboard for monitoring vehicle count, signal timing, and traffic analysis.

1.6 Scope

The scope of the proposed system is limited to a four-way traffic intersection. The current implementation processes recorded traffic videos that simulate CCTV camera feeds for system evaluation and testing. The system employs YOLOv8 for vehicle detection and ByteTrack for vehicle tracking to accurately estimate lane-wise traffic density.

The proposed system focuses on adaptive traffic signal control using density-based optimization and ambulance prioritization. Features such as multi-intersection coordination, cloud-based deployment, IoT integration, and communication with smart city infrastructure are beyond the scope of the current implementation. However, the proposed architecture is scalable and can be integrated with live CCTV cameras and intelligent transportation infrastructure in future deployments after obtaining the required permissions.

1.7 Overview of Proposed System

The proposed system, named **Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision**, is designed to optimize traffic signal timings using computer vision. The system processes traffic videos captured from CCTV cameras. During implementation, recorded traffic videos are used to simulate live CCTV feeds for testing and validation.

OpenCV is utilized for video processing and Region of Interest (ROI) selection. YOLOv8 detects various vehicle categories including cars, motorcycles, buses, trucks, and bicycles, while ByteTrack assigns unique identities to detected vehicles to avoid duplicate counting. Passenger Car Unit (PCU) weights are assigned to different vehicle categories for calculating traffic density.

Based on the computed density, the signal timing algorithm dynamically determines the green signal duration while maintaining predefined minimum and maximum timing limits. In addition, the system provides immediate priority to ambulances by allocating a green signal whenever an emergency vehicle is detected. The processed information, including vehicle count, emergency

vehicles, traffic density, signal timing, and graphical traffic analysis, is displayed through an interactive dashboard developed using React and Tailwind CSS.

1.8 Contributions of the Project

The major contributions of the proposed UrbanEye system are as follows:

- **Vision-Based Traffic Monitoring:** Development of a computer vision-based framework for monitoring traffic conditions at a four-way intersection using video input.
- **Vehicle Detection and Classification:** Integration of the YOLOv8 object detection model to identify and classify different vehicle categories, including cars, motorcycles, buses, trucks, bicycles, and other relevant vehicle types.
- **Vehicle Tracking and Counting:** Integration of the ByteTrack algorithm to maintain vehicle identities across consecutive frames and reduce duplicate vehicle counting.
- **Lane-Wise Traffic Demand Estimation:** Estimation of traffic demand for individual approaches by considering vehicle counts and different vehicle-category weights using the Passenger Car Unit (PCU) concept.
- **Adaptive Signal Timing:** Development of a density-based signal control mechanism that dynamically determines green signal duration according to the estimated traffic demand while maintaining predefined minimum and maximum green-time limits.
- **Emergency Vehicle Prioritization:** Incorporation of ambulance detection and priority handling into the traffic signal control process to support faster passage of emergency vehicles.
- **Traffic Monitoring Dashboard:** Development of an interactive web-based dashboard for displaying vehicle counts, traffic density, signal timing, and other relevant traffic information.
- **Integrated Evaluation Framework:** Implementation and evaluation of the complete detection, tracking, traffic-demand estimation, signal-control, and visualization pipeline using recorded traffic videos that simulate CCTV camera feeds.

1.9 Organization of Report

The remainder of this report is organized as follows. Chapter 2 presents the literature survey of existing traffic signal optimization techniques and identifies research gaps. Chapter 3 describes

the Software Requirements Specification (SRS), including system requirements and use cases. Chapter 4 explains the system design, architecture, and module interactions. Chapter 5 discusses the proposed methodology adopted for vehicle detection, tracking, density calculation, and signal timing optimization. Chapter 6 presents the implementation details along with the developed user interface and experimental results. Chapter 7 summarizes the work through the conclusion, while Chapter 8 discusses the future scope and possible enhancements of the proposed system.

Chapter 2

Literature Survey

2.1 Review Methodology

A systematic literature review was conducted using major digital libraries including IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, ResearchGate, and Google Scholar. The review primarily covered research papers published between 2016 and 2025, with greater emphasis on studies published after 2020. The selected papers focused on intelligent traffic management, adaptive traffic signal control, computer vision, vehicle detection, deep learning, and artificial intelligence-based optimization techniques.

Relevant keywords used during the literature search included *traffic signal optimization*, *YOLO-based vehicle detection*, *adaptive traffic control*, *deep reinforcement learning*, *computer vision*, *vehicle tracking*, *traffic density estimation*, and *intelligent transportation systems*. The reviewed literature helped in understanding existing methodologies, identifying their strengths and limitations, and determining research gaps that motivated the development of the proposed Urban Eye system.

2.2 Traditional and Optimization-Based Traffic Management Systems

Traditional traffic management systems primarily focus on optimizing traffic signal control using mathematical models, optimization algorithms, artificial intelligence, and multi-agent approaches. Researchers have proposed various techniques, including Stackelberg game theory, Braess' Paradox, PLC-based intelligent controllers, lane-changing optimization, mathematical signal timing models, Particle Swarm Optimization (PSO), multi-agent systems, reinforcement learning, Large Language Models (LLMs), and multi-modal data fusion to improve traffic flow, reduce vehicle waiting time, and maximize intersection throughput. These approaches have demonstrated significant improvements over conventional fixed-time traffic signal systems by dynamically adapting signal timings according to traffic conditions. However, many existing methods rely on complex optimization models, simulation environments, dedicated hardware, or

extensive computational resources, making their deployment in real-world urban traffic scenarios challenging. Furthermore, most studies emphasize traffic signal optimization without integrating practical vision-based vehicle detection and tracking techniques for real-time traffic analysis[1][2][3],[5][6][9][10][11][13][17][18][20].

2.3 Computer Vision and Deep Learning-Based Traffic Systems

Recent advancements in computer vision and deep learning have significantly improved the performance of intelligent traffic management systems. Researchers have extensively utilized object detection models such as YOLO, deep reinforcement learning (DRL), machine vision, and image processing techniques for real-time vehicle detection, tracking, counting, and adaptive traffic signal control. These approaches provide higher detection accuracy, faster processing speed, and better adaptability than traditional sensor-based systems. Studies have shown that integrating computer vision with traffic signal optimization can effectively reduce congestion and improve traffic flow under varying traffic conditions. However, several existing systems are evaluated only in simulated environments or focus primarily on vehicle detection without implementing a complete adaptive traffic signal control framework. Furthermore, challenges such as poor lighting conditions, occlusion, adverse weather, and computational requirements continue to affect the reliability of vision-based traffic management systems in real-world scenarios[4][7][8][12][14][15][19].

2.4 IoT and Intelligent Transportation Systems

The integration of Internet of Things (IoT) technologies with Artificial Intelligence (AI) has enabled the development of intelligent transportation systems capable of collecting, processing, and analyzing traffic information from multiple sources. IoT-based traffic management solutions utilize interconnected sensors, communication networks, cloud computing, and AI algorithms to support real-time traffic monitoring, predictive analysis, and adaptive signal control. These systems provide improved scalability, efficient resource utilization, and enhanced decision-making for smart city applications. However, large-scale deployment requires reliable communication infrastructure, robust cybersecurity mechanisms, and significant investment in smart city technologies, which remain major challenges for widespread implementation[16].

2.5 Summary of Literature Survey

Table 2.1 presents a comparative analysis of the major traffic management techniques discussed in the reviewed literature. The comparison summarizes the methodologies adopted by different

researchers, their primary contributions, and the limitations associated with each approach. It provides a comprehensive overview of existing research trends in intelligent traffic signal management and highlights the strengths and weaknesses of current solutions.

Table 2.1: Summary of Literature Survey

Sr.	Author (Year)	Key Contribution	Limitation
1	Kollias (2021)	Stackelberg-based traffic routing optimization	Focuses on routing rather than signal control
2	Singh (2023)	Combines Braess' Paradox and ViT for traffic optimization	Limited intersection-level signal control
3	Wang (2025)	PLC and intelligent algorithms for signal optimization	Requires PLC-based infrastructure
4	Azevedo (2024)	Compares YOLO detectors and trackers for edge-based ADAS	Focuses on ADAS rather than traffic signals
5	Shen (2026)	Optimizes lane changes and vehicle trajectories at intersections	Requires detailed vehicle trajectory information
6	Harantová (2021)	Mathematical model for traffic-light timing optimization	Relies mainly on traffic-flow assumptions
7	Liang (2019)	Deep reinforcement learning for adaptive signal control	Requires extensive training and simulation

Table 2.1 – *Continued*

Sr.	Author (Year)	Key Contribution	Limitation
8	Ayegbusi (2025)	YOLO-based traffic density estimation and signal control	Limited tracking and emergency-vehicle handling
9	Quan (2009)	Joint optimization of traffic organization and signal control	Lacks modern vision-based traffic detection
10	Withanawasam (2017)	Multi-agent traffic control using CCTV-based density detection	Coordination increases system complexity
11	Lakshay (2025)	AI-based traffic signal optimization for congestion control	Limited vehicle tracking implementation
12	Tahmid (2017)	Canny-based vehicle density estimation for signal control	Less robust than deep-learning detection
13	Manandhar (2018)	PSO-based adaptive traffic-light control	Lacks integrated vision-based tracking
14	Vyas (2025)	Machine-vision-based vehicle detection and speed estimation	Does not focus on adaptive signal control
15	Sakhuja (2023)	Computer vision and ML for intelligent traffic management	Limited adaptive signal-control implementation

Table 2.1 – *Continued*

Sr.	Author (Year)	Key Contribution	Limitation
16	Ouallane et al. (2021)	Reviews IoT and AI-based traffic management solutions	Does not implement a unified system
17	Li (2026)	Multi-modal data fusion for signal control and path planning	High computational complexity
18	Singh (2025)	LLM and reinforcement learning for traffic optimization	Practical deployment remains challenging
19	Song (2019)	Deep-learning-based vehicle detection and counting	Focuses on highways rather than intersections
20	Eom and Kim (2020)	Reviews intersection traffic signal control methods	Does not provide an integrated implementation

2.6 Research Gaps

The literature review highlights several limitations in existing traffic management systems.

- Most vision-based traffic management systems primarily focus on vehicle detection and counting without integrating adaptive traffic signal control.
- Deep Reinforcement Learning-based approaches are generally validated only in simulation environments and are rarely evaluated using actual traffic video captured from CCTV cameras.
- Several traffic optimization systems require expensive hardware such as inductive loop detectors, IoT sensors, or dedicated communication infrastructure, making large-scale deployment costly.

- Many existing methods experience reduced detection accuracy under challenging conditions such as poor illumination, occlusion, adverse weather, and heavy traffic congestion.
- Only a limited number of studies consider emergency vehicle prioritization while simultaneously maintaining fair traffic signal allocation for normal traffic.
- Most existing systems provide limited visualization capabilities and lack interactive dashboards for real-time traffic monitoring and analysis.
- Very few studies combine vehicle detection, vehicle tracking, weighted density estimation, adaptive signal timing, emergency vehicle prioritization, and traffic analytics within a single integrated framework.

To address these research gaps, the proposed **UrbanEye: Vision-Based Automated Traffic Optimization Using Computer Vision** system integrates YOLOv8-based vehicle detection, ByteTrack-based vehicle tracking, weighted traffic density estimation, adaptive signal timing, ambulance prioritization, and an interactive dashboard into a unified traffic management framework. The current implementation uses recorded traffic videos to simulate CCTV camera feeds, while the overall architecture is designed to support direct integration with live CCTV infrastructure in future deployments.

Chapter 3

Analysis / Software Requirements Specification (SRS)

3.1 Existing System

Conventional traffic signal control systems are primarily based on fixed-time signal scheduling or sensor-based adaptive traffic management. Fixed-time traffic signals operate according to predefined timing schedules developed using historical traffic data. Since these systems do not consider current traffic conditions, they often allocate equal signal duration to all lanes regardless of the actual vehicle density.

Sensor-based traffic management systems improve upon fixed-time control by employing devices such as inductive loop detectors, infrared sensors, and magnetic sensors to estimate traffic flow and dynamically adjust signal timings. Although these systems provide better performance, they require expensive hardware installation, periodic maintenance, and modifications to existing road infrastructure. Consequently, their large-scale deployment becomes costly, particularly in developing countries.

3.1.1 Limitations of Existing System

- Traffic signals are unable to adapt to changing traffic conditions.
- Equal signal duration is allocated irrespective of vehicle density.
- Installation and maintenance costs are high due to specialized sensors.
- Existing systems require physical modifications to road infrastructure.
- Limited support for emergency vehicle prioritization.
- Poor utilization of road capacity during uneven traffic distribution.

3.2 Proposed System

The proposed system, **Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision**, is designed to overcome the limitations of traditional traffic signal systems by utilizing computer vision and deep learning techniques for adaptive traffic signal control.

The current implementation processes recorded traffic videos that simulate CCTV camera feeds. The architecture is designed so that it can be directly integrated with live CCTV cameras in future deployments after obtaining the required infrastructure and permissions.

Initially, OpenCV captures video frames and performs preprocessing. A Region of Interest (ROI) is defined for each lane so that only relevant road areas are analyzed. Vehicle detection is performed using the YOLOv8 object detection model, which identifies vehicles such as cars, motorcycles, buses, trucks, and bicycles. ByteTrack assigns a unique identity to each detected vehicle, ensuring that vehicles are counted only once while moving through consecutive frames.

Each detected vehicle is assigned a predefined Passenger Car Unit (PCU) weight according to its category. The weighted vehicle count is then used to calculate lane-wise traffic density. Based on the calculated density, the adaptive signal timing algorithm dynamically determines the green signal duration while maintaining predefined minimum and maximum limits. Lanes without vehicles are skipped to avoid unnecessary waiting time.

The proposed system also supports ambulance prioritization. Whenever an ambulance is detected, the corresponding lane immediately receives signal priority to facilitate faster emergency response.

Finally, all processed information is displayed through an interactive dashboard developed using React and Tailwind CSS. The dashboard presents four lane-wise video streams, vehicle count, calculated traffic density, signal timing, signal status, and graphical traffic analysis.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The proposed system is technically feasible because it is developed using widely adopted open-source technologies including Python, OpenCV, YOLOv8, ByteTrack, React, and Tailwind CSS. These technologies support efficient computer vision processing and are compatible with standard computing platforms. The modular architecture also simplifies future upgrades and integration with live CCTV systems.

3.3.2 Economical Feasibility

The proposed system is economically feasible because it utilizes existing CCTV infrastructure instead of requiring dedicated roadside sensors or additional hardware. Since the software is primarily based on open-source frameworks, implementation and maintenance costs are significantly reduced compared with conventional sensor-based traffic management systems.

3.3.3 Operational Feasibility

The proposed system provides a user-friendly interface for monitoring traffic conditions and signal timing decisions. The dashboard enables traffic operators to easily visualize vehicle count, traffic density, signal status, and statistical charts. Minimal operator training is required because the system performs all computational tasks automatically.

3.4 System Requirements

3.4.1 Functional Requirements

- The system shall process recorded traffic videos that simulate CCTV camera feeds.
- The system shall detect vehicles using the YOLOv8 object detection model.
- The system shall track detected vehicles using the ByteTrack algorithm.
- The system shall classify vehicles into different categories.
- The system shall calculate weighted traffic density for each lane.
- The system shall dynamically allocate green signal time based on traffic density.
- The system shall maintain predefined minimum and maximum green signal durations.
- The system shall skip lanes without vehicles.
- The system shall provide priority to ambulances.
- The system shall display vehicle count, traffic density, signal timing, signal status, and traffic analysis through the dashboard.

3.4.2 Non-Functional Requirements

- The system should provide near real-time performance.
- The system should maintain high vehicle detection accuracy.

- The system should be reliable during continuous operation.
- The system should be scalable for future expansion.
- The system should be easy to maintain and upgrade.
- The system should provide an intuitive and user-friendly interface.

3.4.3 Software Requirements

- Python
- OpenCV
- YOLOv8
- ByteTrack
- Websocket
- Flask
- React
- Tailwind CSS
- Chart.js
- Visual Studio Code

3.4.4 Hardware Requirements

- Intel Core i5 processor or above
- Minimum 8 GB RAM (16 GB recommended)
- NVIDIA GPU (recommended for faster YOLOv8 inference)
- Minimum 5 GB available storage
- Webcam or CCTV camera (for future real-time deployment)
- Display monitor with minimum 1920×1080 resolution

3.4.5 Performance Requirements

- The system should detect and classify vehicles with high accuracy.
- The system should calculate lane-wise traffic density within a few seconds after processing each lane.
- The signal timing should be updated immediately after density calculation.
- The dashboard should display updated vehicle count and signal timing without noticeable delay.
- The system should support continuous processing of traffic videos with stable performance.

3.5 Use Case

The proposed system consists of a single primary actor, namely the **Administrator**. The administrator monitors the traffic conditions through the dashboard, observes lane-wise vehicle density, views traffic analysis charts, monitors signal timings, and supervises signal status. The system automatically performs vehicle detection, tracking, density calculation, and signal timing optimization without requiring manual intervention. Additionally, the administrator can monitor emergency vehicle detection and verify signal changes through the dashboard.

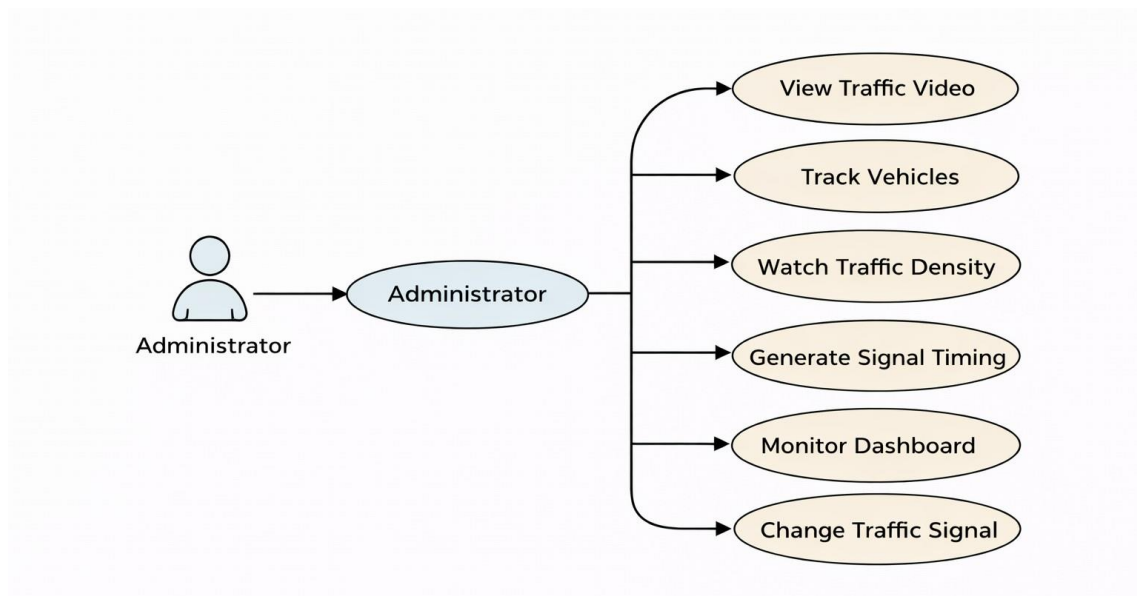


Figure 3.1: Use Case Diagram of Vision-Based Traffic Signal Optimization System

3.6 Constraints

- The performance of the system depends on the quality, resolution, and placement of the traffic camera.

- Vehicle detection accuracy may decrease under poor lighting conditions, heavy rain, fog, or severe occlusion.
- The current implementation is evaluated using recorded traffic videos that simulate CCTV camera feeds.
- The proposed system currently supports only a single four-way intersection.
- Ambulance detection accuracy depends on the quality of the trained YOLOv8 model.
- The system assumes a fixed camera position and Region of Interest (ROI) for each lane.

3.7 Advantages of Proposed System

- Dynamically allocates traffic signal duration according to real-time traffic density.
- Reduces unnecessary vehicle waiting time and traffic congestion.
- Improves overall traffic flow efficiency.
- Eliminates the need for expensive roadside sensors by utilizing computer vision.
- Cost-effective because it can utilize existing CCTV infrastructure.
- Provides priority to emergency vehicles such as ambulances.
- Prevents duplicate vehicle counting using the ByteTrack algorithm.
- Displays traffic information, vehicle analysis, and signal timing through an interactive dashboard.
- Can be integrated with live CCTV cameras and expanded to multi-intersection traffic management in future implementations.

3.8 Summary

This chapter presented the Software Requirements Specification (SRS) of the proposed **Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision** system. It discussed the limitations of existing traffic management systems, described the proposed solution, evaluated its technical, economical, and operational feasibility, and specified the functional and non-functional requirements. Furthermore, software and hardware requirements, performance requirements, the use case diagram, system constraints, and the major advantages of the proposed system were presented.

Chapter 4

System Design

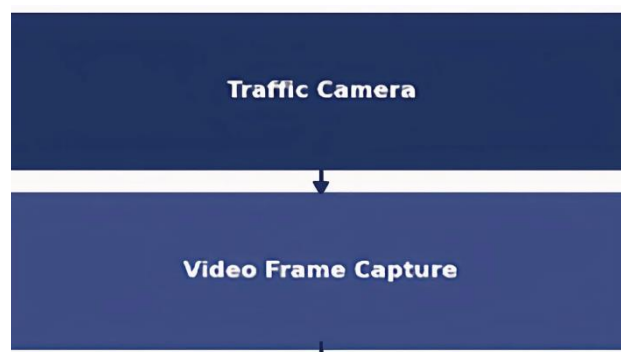
4.1 System Architecture

The proposed system follows a modular pipeline architecture in which recorded traffic videos, representing CCTV camera feeds, are processed sequentially to generate adaptive traffic signal timings. Unlike conventional sensor-based systems, the proposed approach relies on computer vision techniques to estimate traffic density and optimize signal control using existing camera infrastructure.

The processing pipeline begins with video acquisition, where traffic videos from a four-way intersection are captured and processed using OpenCV. A Region of Interest (ROI) is defined to analyze only the traffic lanes. Vehicles are detected using the YOLOv8 model, while ByteTrack assigns a unique identity to each detected vehicle to prevent duplicate counting.

The detected vehicles are used to calculate lane-wise traffic density based on predefined Passenger Car Unit (PCU) weights. The signal timing module then computes the green signal duration according to the calculated density. If an ambulance is detected, the system immediately grants signal priority to the corresponding lane. Finally, the processed information, including vehicle count, signal status, and traffic statistics, is displayed on the web dashboard for monitoring and analysis.

The modular architecture improves system maintainability, scalability, and future extensibility. It also provides a foundation for integrating live CCTV cameras and supporting multi-intersection traffic management in future implementations.



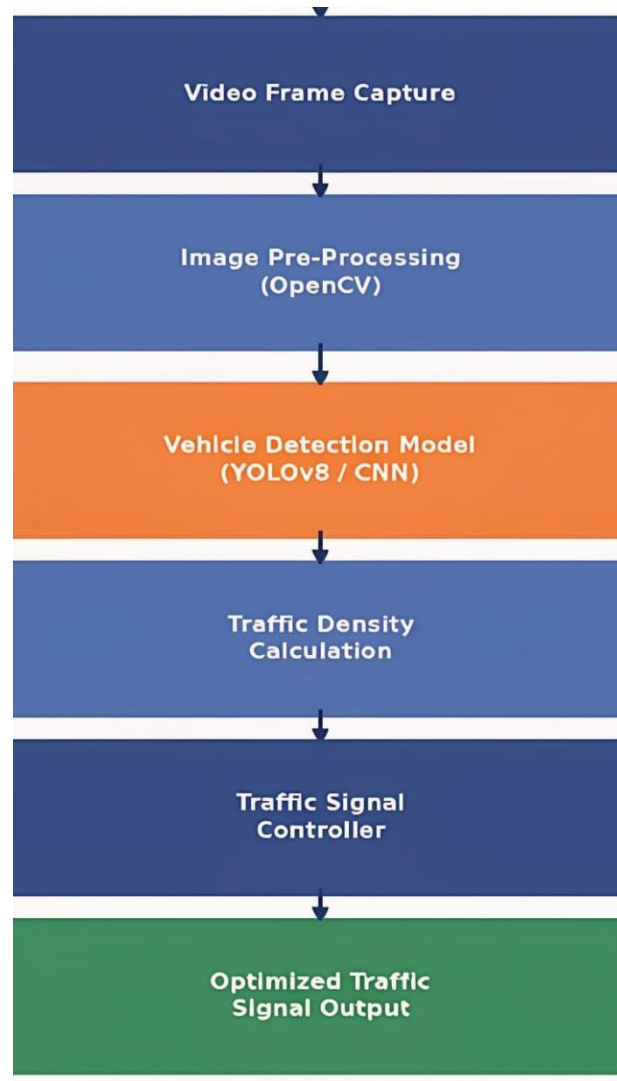


Figure 4.1: Overall System Architecture

4.2 Module Design

The proposed Urban Eye system is divided into several functional modules. Each module performs a dedicated task within the traffic optimization pipeline and exchanges information with other modules to produce the final signal timing decision.

4.2.1 Video Acquisition Module

The Video Acquisition Module is responsible for receiving the traffic videos used by the proposed system. The current implementation uses recorded traffic videos that simulate CCTV camera feeds. Each video corresponds to one approach of the four-way intersection. OpenCV reads the video, extracts individual frames, and forwards them for further processing.

Functions

- Accept recorded traffic videos as CCTV simulation.

- Extract video frames using OpenCV.
- Maintain continuous frame processing.
- Forward frames to the preprocessing stage.

4.2.2 Vehicle Detection Module

The Vehicle Detection Module identifies vehicles present within each frame using the YOLOv8 object detection model. Before detection, every frame is resized to the required input resolution to improve inference consistency. A predefined Region of Interest (ROI) is applied so that only vehicles travelling within the traffic lanes are considered during processing.

The model detects and classifies five vehicle categories, namely cars, buses, trucks, motorcycles, and bicycles. For every detected vehicle, the model returns the class label, confidence score, and bounding box coordinates.

Functions

- Detect vehicles using the YOLOv8 model.
- Apply ROI filtering before counting vehicles.
- Classify vehicles into predefined categories.
- Generate bounding boxes and confidence scores.
- Forward detection results to the tracking module.

4.2.3 Vehicle Tracking Module

The Vehicle Tracking Module uses the ByteTrack algorithm to assign a unique identification number to every detected vehicle. Instead of counting detections independently in every frame, ByteTrack follows the movement of each vehicle across consecutive frames and maintains a persistent identity throughout its journey.

Tracking prevents the same vehicle from being counted multiple times while it remains inside the Region of Interest. This significantly improves counting accuracy and provides reliable vehicle statistics for density estimation.

Functions

- Assign a unique ID to every detected vehicle.
- Track vehicle movement across consecutive frames.

- Prevent duplicate vehicle counting.
- Provide accurate lane-wise vehicle statistics.

4.2.4 Traffic Density Estimation Module

The Traffic Density Estimation Module computes the traffic density for each lane using the detected and tracked vehicles. Instead of treating every vehicle equally, the system assigns a predefined Passenger Car Unit (PCU) weight to each vehicle category to represent its relative impact on traffic flow. Larger vehicles such as buses and trucks contribute more to traffic density than smaller vehicles such as motorcycles and bicycles.

The density of each lane is calculated by summing the weighted values of all detected vehicles within the Region of Interest (ROI). The calculated density is then forwarded to the signal timing module for green signal allocation.

Functions

- Calculate lane-wise vehicle density.
- Apply predefined PCU weights.
- Generate weighted traffic density.
- Forward density values to the signal timing module.

4.2.5 Signal Timing Module

The Signal Timing Module dynamically determines the green signal duration based on the calculated traffic density. Each lane receives green time proportional to its traffic density, thereby improving traffic flow and reducing unnecessary waiting time.

The proposed system uses a density-based mathematical model with predefined constants. The green signal duration is restricted within minimum and maximum limits to ensure fair signal allocation among all lanes. If a lane contains no vehicles, it is skipped and the signal is directly assigned to the next lane containing traffic.

In addition, the module continuously checks for ambulance detection. Whenever an ambulance is detected, the normal signal sequence is temporarily overridden and immediate green signal priority is provided to the corresponding lane.

Functions

- Receive lane-wise density values.

- Calculate green signal duration.
- Apply minimum and maximum signal limits.
- Skip empty lanes.
- Provide ambulance priority.
- Update signal status on the dashboard.

4.2.6 Helmet Detection Module

The Helmet Detection Module is responsible for identifying whether two-wheeler riders are wearing helmets. The module processes the video frames using a trained computer vision model to detect riders and helmets. The detected results are used to identify helmet compliance and display the corresponding information for traffic monitoring.

Functions

- Detect two-wheeler riders from processed video frames.
- Identify helmet and non-helmet cases using the trained detection model.
- Associate detected helmets with the corresponding riders.
- Display helmet detection results for monitoring and analysis.

4.2.7 Dashboard Module

The Dashboard Module provides an interactive graphical interface for monitoring traffic conditions and system performance. It is developed using React and Tailwind CSS and communicates with the backend to display real-time information generated by the processing modules.

The dashboard consists of multiple pages, including the landing page, monitoring dashboard, home page, and analytics dashboard. It displays vehicle counts, lane-wise traffic density, signal timings, emergency vehicle status, and traffic analysis charts. The interface enables users to observe traffic conditions and verify signal decisions in a simple and user-friendly manner.

4.3 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates the movement of information throughout the proposed system. Traffic videos are first provided as input to the video acquisition module. The extracted

frames are processed using YOLOv8 for vehicle detection and ByteTrack for vehicle tracking. The tracked vehicle information is used to calculate lane-wise traffic density, which is then supplied to the signal timing module for determining the appropriate green signal duration. Finally, all generated information is displayed through the dashboard for monitoring and analysis.

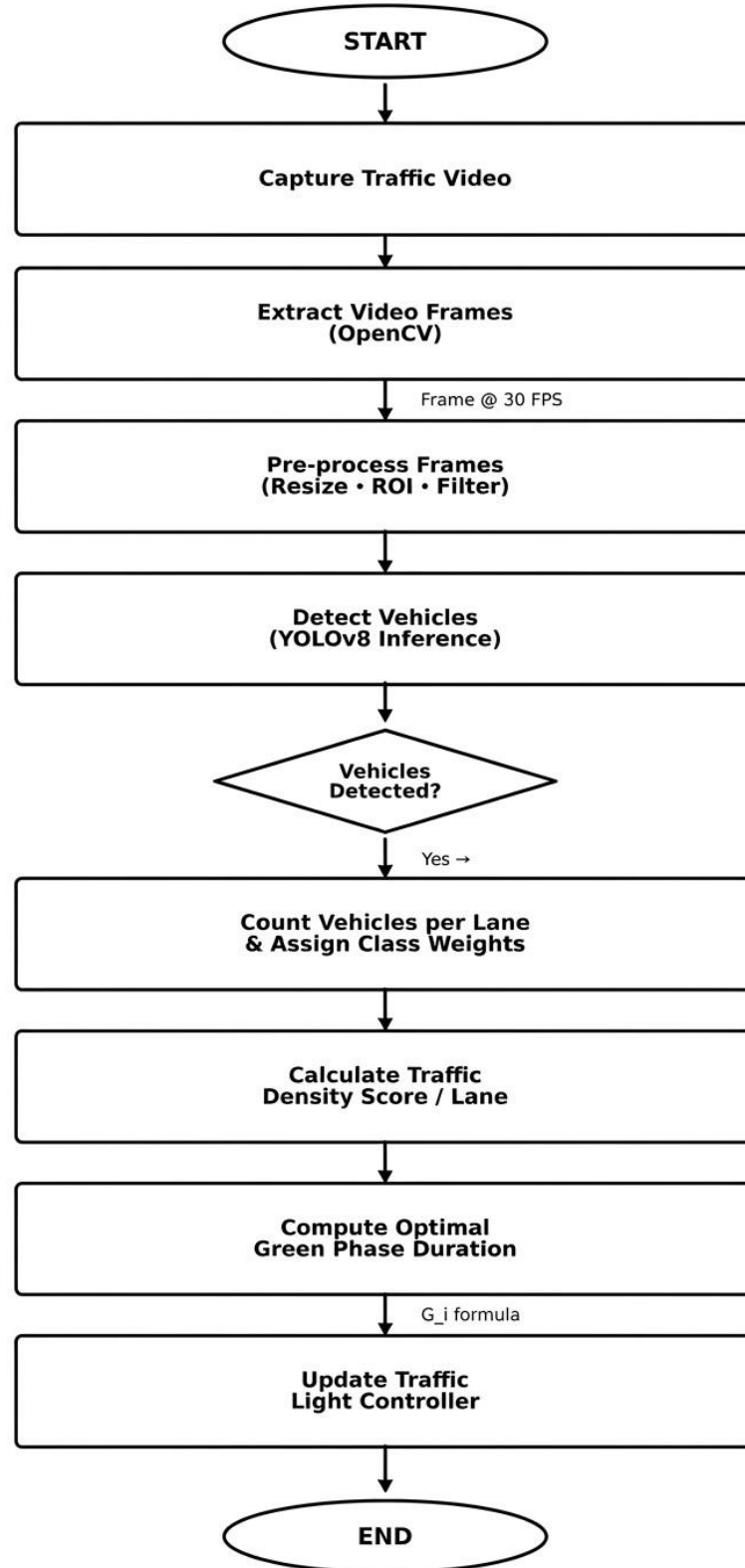


Figure 4.2: Data Flow Diagram of the Proposed System

4.4 Overall System Workflow

The overall workflow of the proposed system consists of a sequence of processing stages that convert traffic video into intelligent signal control decisions. Each stage performs a dedicated task and passes the processed information to the subsequent module.

1. Capture recorded traffic videos representing CCTV camera feeds.
2. Extract individual video frames using OpenCV.
3. Apply Region of Interest (ROI) to focus on traffic lanes.
4. Detect vehicles using the YOLOv8 object detection model.
5. Track detected vehicles using the ByteTrack algorithm.
6. Calculate weighted traffic density using predefined PCU weights.
7. Compute green signal duration for each lane.
8. Check for ambulance detection and provide emergency priority if required.
9. Display vehicle count, signal timing, and analytical information on the dashboard.

4.5 User Interface Design

The user interface of the proposed system is designed to provide an intuitive and interactive platform for monitoring traffic conditions and signal operations. The frontend is developed using React, while Tailwind CSS is used to create a responsive and modern interface. The dashboard communicates with the backend to display vehicle statistics, signal timing information, and traffic analysis results.

The interface is divided into four major sections: Landing Page, Home Dashboard, Live Monitoring Dashboard, and Analytics Dashboard. Each page is designed to perform a specific function while maintaining a consistent layout and user experience.

4.5.1 Landing Page

The landing page serves as the entry point of the Urban Eye system. It introduces the proposed traffic optimization platform and provides quick navigation to the monitoring dashboard and analytics section. It also displays key system statistics, including total vehicles processed, average waiting time reduction, emergency vehicle handling, and active monitoring nodes.

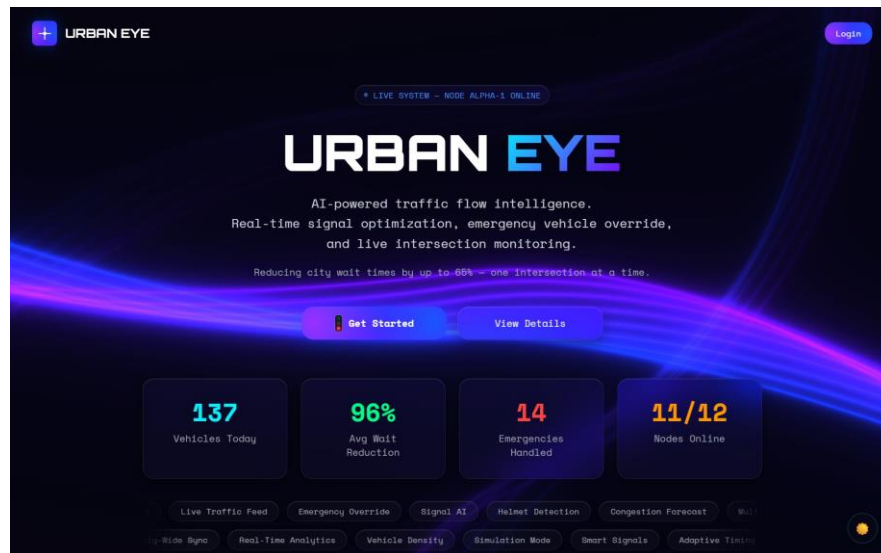


Figure 4.3: Landing Page of Urban Eye

4.5.2 Home Dashboard

The Home Dashboard provides an entry point for accessing the Urban Eye traffic monitoring system. It allows users to select between two operating modes: Live Monitoring and Simulation. Live Monitoring enables users to upload one or all four traffic-lane videos and view real-time vehicle detection and monitoring, while Simulation provides a demonstration of the system using simulated traffic data.

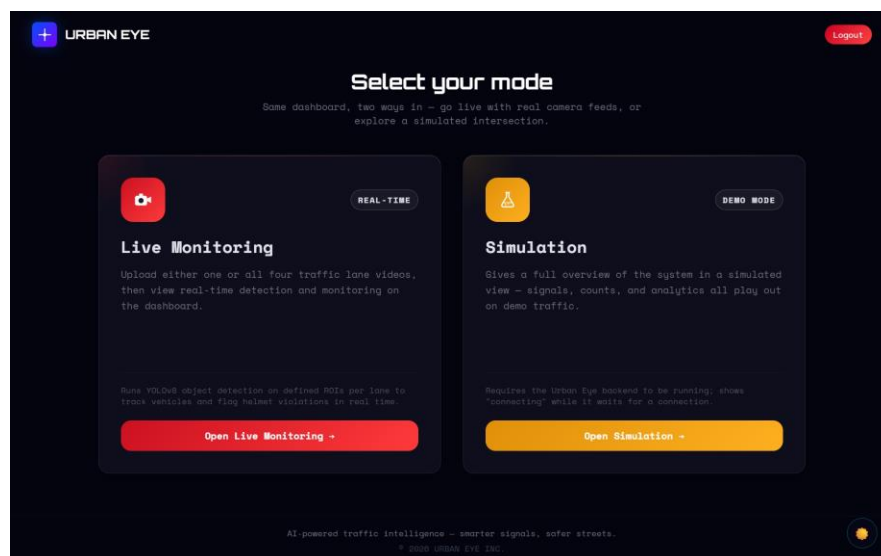


Figure 4.4: UrbanEye Dashboard Before Video Input

4.5.3 Monitor Dashboard

The Monitor Dashboard provides a unified interface for observing and managing the Urban Eye traffic monitoring system. It consists of two modes: Simulation Mode and Live Mode, allowing the system to be evaluated using simulated traffic or monitored using actual traffic video feeds.

1. Simulation Mode

In Simulation Mode, the dashboard represents a four-way intersection with simulated vehicles and traffic signals. It displays key metrics such as live vehicle count, traffic efficiency, average waiting time, active lane, vehicle counts by direction and type, signal timing, and traffic throughput. It also includes an event log for emergency events and signal overrides, an AI-based congestion forecast, weather information, and node-health status for connected cameras, signal controllers, communication modules, and sensors.

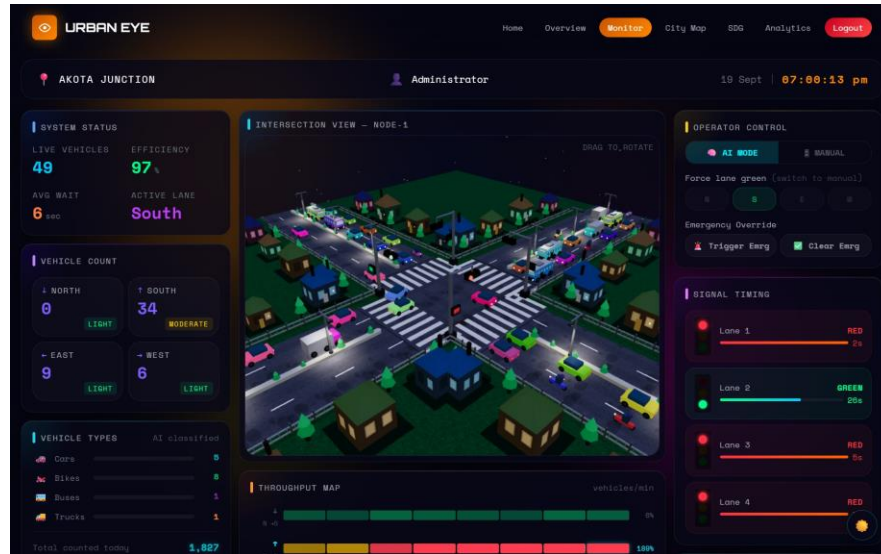


Figure 4.5: Monitor dashboard of simulation mode

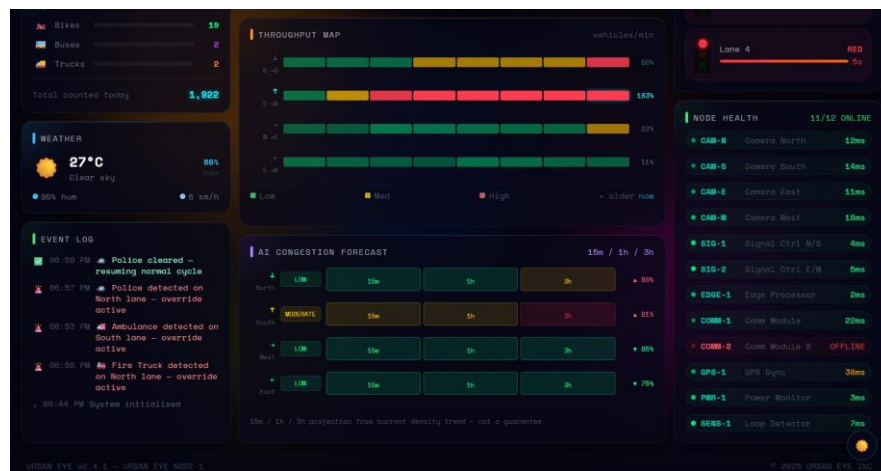


Figure 4.6: Information shown on dashboard

2. Live Mode

In Live Mode, the Monitor Dashboard processes uploaded traffic video feeds representing the four approaches of an intersection. Each feed is displayed with real-time vehicle detection and bounding boxes, while the corresponding traffic signal indicators show the current signal state for

each lane. The dashboard provides operator controls for AI and manual signal operation, including lane selection, green-time allocation, and emergency signal control.

The dashboard provides detailed real-time traffic information through multiple monitoring panels. The System Status panel displays the number of currently detected vehicles, traffic efficiency, average waiting time, and the active lane. The Signal Timing panel displays the remaining signal duration and current signal state for each lane. In addition, the Throughput Map visualizes the traffic flow across different directions, while the AI Congestion Forecast provides projected traffic conditions for different time intervals. These modules work together to provide a centralized view of the current traffic situation and support dynamic traffic signal management.

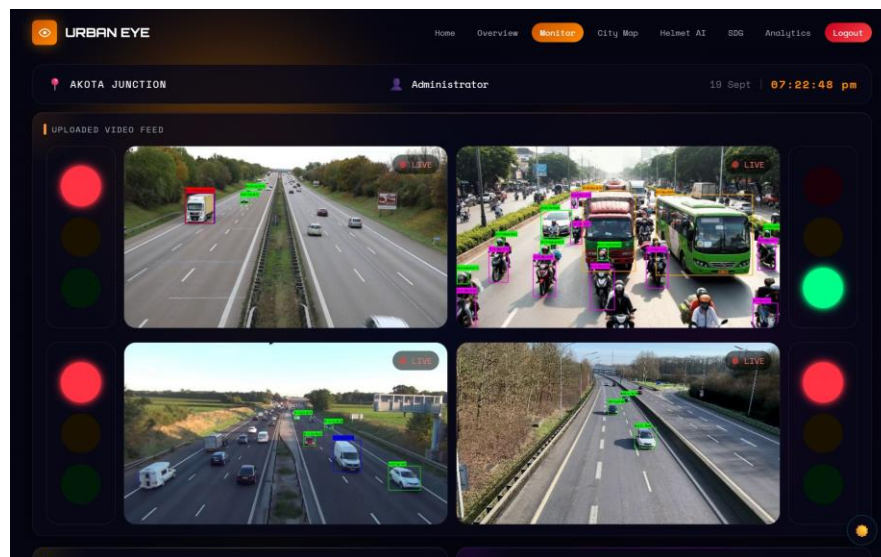


Figure 4.7: Monitor dashboard of Live mode

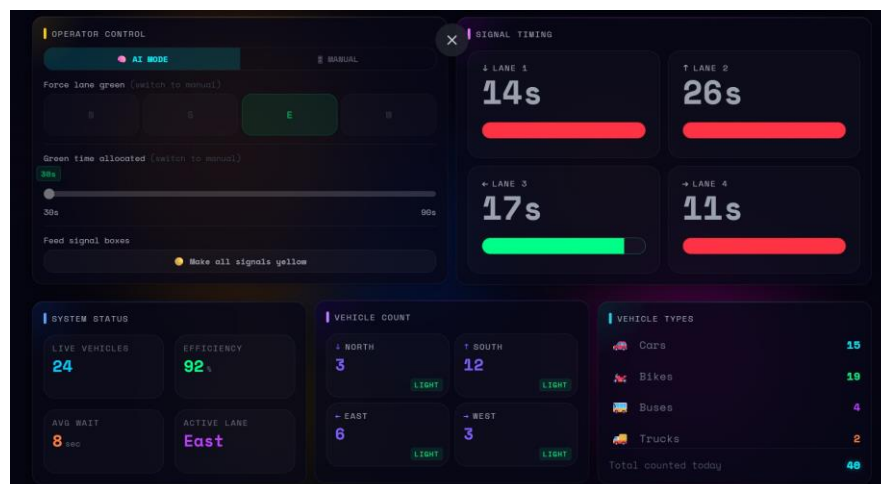


Figure 4.8: Information shown on dashboard

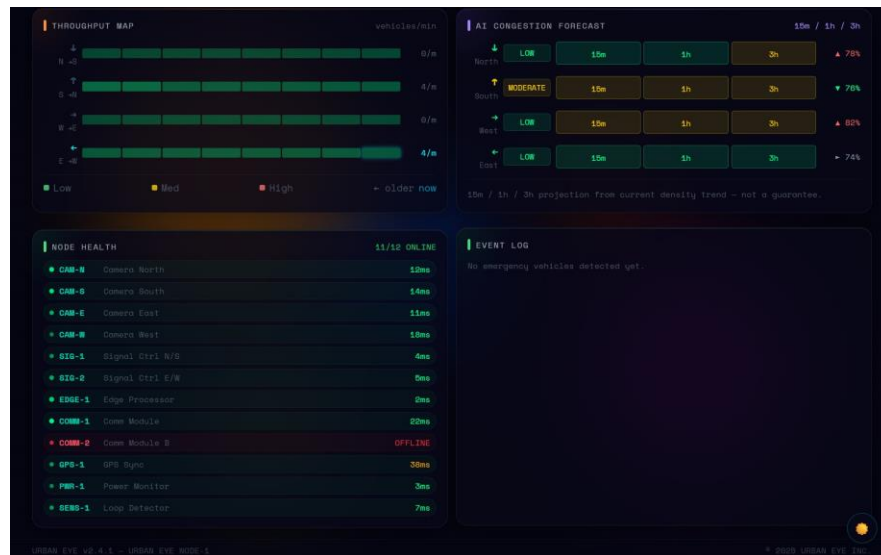


Figure 4.9: Additional Info shown on dashboard

4.5.4 City Map Dashboard

The City Map Dashboard provides a geographical overview of the Urban Eye traffic and emergency management system. It displays the monitored junction and surrounding areas on an interactive city map, along with color-coded markers representing traffic conditions and important locations. The dashboard includes an Emergency Fleet panel showing the availability of ambulances, fire trucks, and other emergency vehicles, with options to select a hospital and dispatch an emergency unit toward the required lane approach. It also provides current information and a Green Wave feature that allows predefined routes to be selected and coordinated signal timing to be activated, helping emergency vehicles move through monitored intersections with reduced traffic delays.

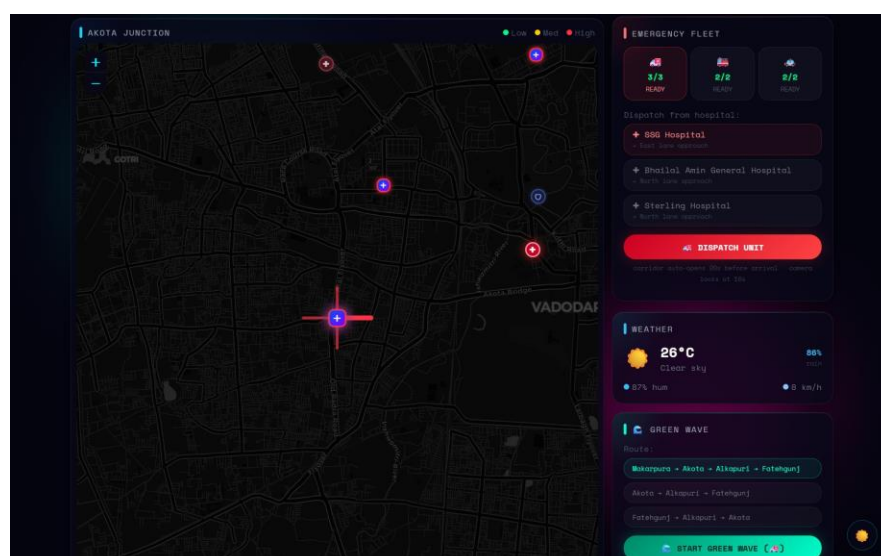


Figure 4.10: City map dashboard

4.5.5 Helmet Dashboard

The Helmet Dashboard provides a centralized view for monitoring helmet compliance across multiple traffic camera feeds. It displays four camera views with real-time vehicle and rider detection, including bounding boxes around detected objects. The dashboard also presents key statistics such as the total number of riders, riders wearing helmets, riders without helmets, and the overall helmet compliance percentage. This enables the system to identify helmet violations and provide a clear overview of road-safety compliance based on the monitored traffic footage.



Figure 4.11: Helmet Detection Dashboard

4.5.6 Sustainability (SDG) Dashboard

The SDG Dashboard presents the sustainability impact of the Urban Eye system by measuring environmental, social, and economic benefits achieved through optimized traffic management. It displays key indicators such as **CO₂ emissions avoided, fuel saved from idling, commuter hours saved, and tree-year equivalent**. The dashboard also visualizes CO₂ savings across traffic cycles and different monitored centers, along with air-quality information including AQI and visibility. In addition, the economic impact section presents estimated fuel-cost and commuter-value savings, providing an overall view of how intelligent traffic signal optimization can contribute to sustainable transportation and urban development.

These indicators help demonstrate the broader impact of the proposed system beyond traffic flow optimization. By reducing unnecessary idling and waiting time, Urban Eye can contribute to lower fuel consumption and emissions while improving commuter efficiency. The collected metrics provide a measurable basis for evaluating the system's contribution toward sustainable and smarter urban mobility.

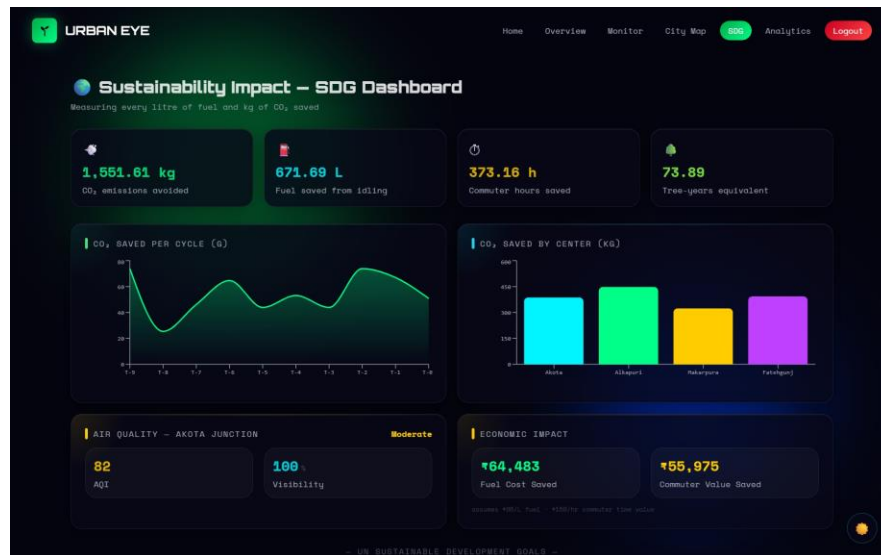


Figure 4.12: Sustainability (SDG) Dashboard

4.5.7 Analytics Dashboard

The Analytics Dashboard provides a detailed analysis of traffic performance at the monitored junction over selected time periods. It presents key metrics such as total vehicles, average waiting time, waiting-time reduction, peak traffic hours, and the number of emergency incidents. The dashboard includes visualizations for traffic volume, a comparison between urban eye AI and fixed-timer signal performance, lane-wise traffic performance, and the reduction in waiting time and fuel consumption. It also provides incident response-time analysis and an emergency incident log containing details such as incident time, vehicle type, affected lane, signal override duration, and resolution status.

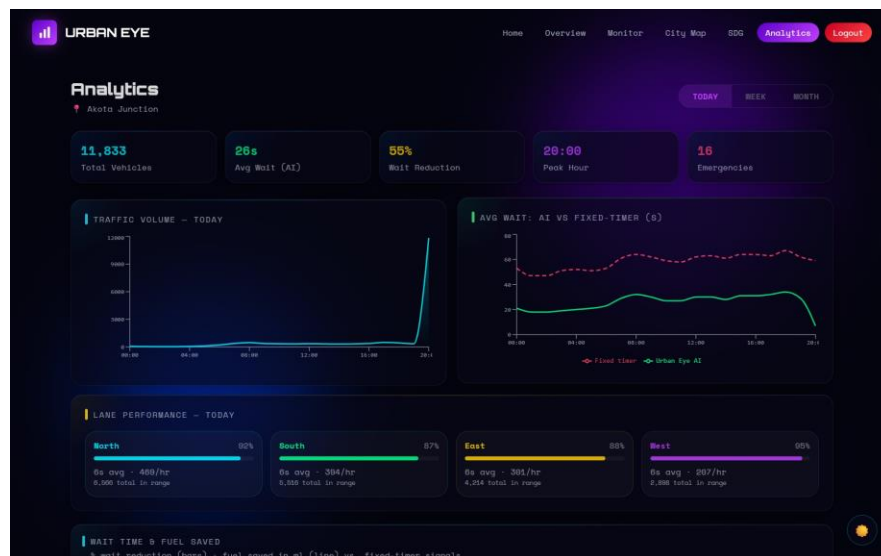


Figure 4.13: Analytics Dashboard (Overview)

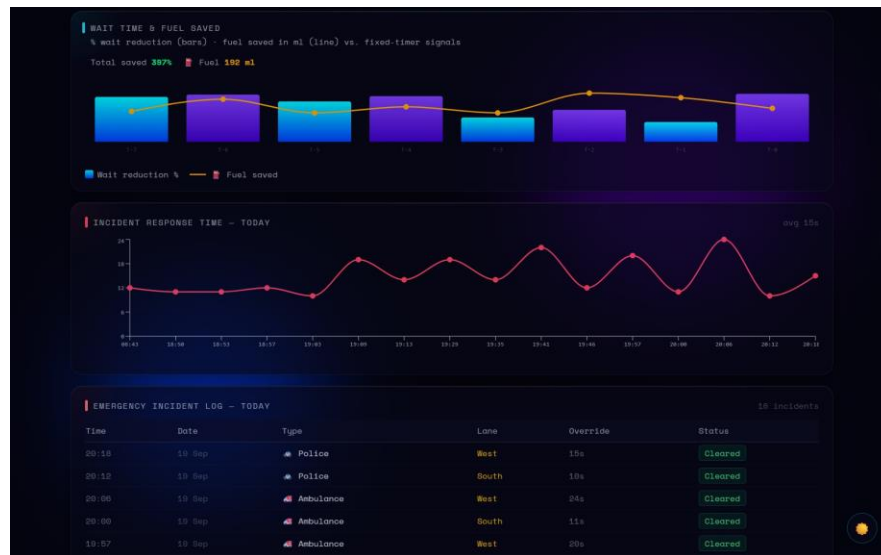


Figure 4.14: Analytics Dashboard (Performance Analysis)

4.6 Technology Stack

The proposed Urban Eye system is developed using a combination of modern software technologies to achieve efficient traffic monitoring and signal optimization. Python is used as the primary programming language for backend development, while OpenCV, YOLOv8, and ByteTrack are utilized for video processing, vehicle detection, and object tracking, respectively. Flask provides the communication between the backend and frontend, whereas React and Tailwind CSS are used to develop a responsive and interactive web dashboard. The complete technology stack is summarized in Table 4.1.

Table 4.1: Technology Stack Used in the Proposed System

Component	Technology	Purpose
Programming Language	Python	Backend Development
Computer Vision	OpenCV	Video Processing
Object Detection	YOLOv8	Vehicle Detection
Object Tracking	ByteTrack	Vehicle Tracking
Frontend	React	User Interface
Styling	Tailwind CSS	Dashboard Design
Charts	Chart.js	Data Visualization
API Framework	Flask	Backend API

4.7 Design Considerations

Several design considerations were taken into account during the development of the proposed Urban Eye system to ensure efficiency, scalability, and ease of deployment.

4.7.1 Modularity

The system follows a modular architecture in which each functional component performs an independent task. Separate modules are implemented for video acquisition, vehicle detection, vehicle tracking, traffic density estimation, signal timing calculation, and dashboard visualization. This modular design simplifies system maintenance, debugging, and future enhancements.

4.7.2 Scalability

The proposed architecture is designed to support future expansion. Although the current implementation is limited to a single four-way intersection using recorded traffic videos, the same framework can be extended to process multiple intersections and integrate with live CCTV cameras without major architectural changes.

4.7.3 Performance

To achieve efficient processing, the system combines the YOLOv8 object detection model with the ByteTrack tracking algorithm. Region of Interest (ROI) selection further reduces unnecessary computations by processing only relevant road areas, thereby improving overall system performance.

4.7.4 Cost Effectiveness

Unlike conventional traffic management systems that require dedicated roadside sensors, the proposed system utilizes existing CCTV infrastructure. This significantly reduces installation and maintenance costs while making deployment more practical for smart city applications.

4.7.5 Reliability

Vehicle tracking using ByteTrack minimizes duplicate counting and improves the reliability of traffic density estimation. Additionally, predefined minimum and maximum signal timing limits ensure fair signal allocation even under varying traffic conditions.

4.7.6 Maintainability

The software is developed using well-established open-source technologies including Python, OpenCV, YOLOv8, Flask, React, and Tailwind CSS. The modular implementation and organized project structure simplify future maintenance, debugging, and feature integration.

4.8 Summary

This chapter presented the system design of the proposed Urban Eye traffic optimization system. The overall system architecture, functional modules, data flow, processing workflow, user interface, and design considerations were discussed in detail. The modular architecture enables efficient vehicle detection, tracking, traffic density estimation, adaptive signal timing, and dashboard visualization while supporting future scalability and integration with live CCTV infrastructure. The design serves as the foundation for the implementation and experimental evaluation presented in the next chapter.

Chapter 5

Methodology

5.1 Overview

The proposed **Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision** system follows a pipeline-based methodology to monitor traffic conditions and dynamically optimize traffic signal timings. The current implementation processes recorded traffic videos that simulate CCTV camera feeds from a four-way intersection. The methodology integrates computer vision and deep learning techniques to detect, track, and count vehicles before calculating lane-wise traffic density.

Initially, the input video is processed using OpenCV to extract individual frames. A predefined Region of Interest (ROI) is applied to focus only on the traffic lane. Vehicles are then detected using the YOLOv8 object detection model and tracked using the ByteTrack algorithm to prevent duplicate counting. The detected vehicles are assigned predefined Passenger Car Unit (PCU) weights for density estimation. Based on the calculated density, the system dynamically determines the green signal duration for each lane. If an ambulance is detected, the normal signal sequence is temporarily overridden to provide immediate priority. Finally, all generated information is displayed through an interactive dashboard for monitoring and analysis.

5.2 System Workflow

The proposed system operates through a sequence of processing stages, where each stage performs a dedicated task before passing its output to the next stage. The complete workflow of the proposed system is summarized in Table 5.1.

Table 5.1: System Workflow Steps

Step	Description
1	Capture and process video frames using OpenCV.

Table 5.1: Continued

Step	Description
2	Define the Region of Interest (ROI) for each traffic lane.
3	Detect vehicles using the YOLOv8 object detection model.
4	Track detected vehicles using the ByteTrack algorithm.
5	Count lane-wise vehicles while avoiding duplicate counting.
6	Calculate weighted traffic density using PCU values.
7	Determine green signal duration based on traffic density.
8	Give priority to emergency vehicles when detected.
9	Display traffic information on the monitoring dashboard.

5.3 Region of Interest (ROI) Selection

To improve processing efficiency and eliminate unnecessary background information, the proposed system uses a manually defined Region of Interest (ROI) for each traffic lane. During the initial execution, the user selects the lane boundaries by clicking multiple points on the video frame. These selected coordinates are stored in a configuration file and automatically reused whenever the same video is processed again.

Restricting vehicle detection to the selected ROI minimizes false detections caused by parked vehicles, pedestrians, or nearby roads while reducing computational overhead. This approach improves detection accuracy and ensures that only vehicles travelling within the designated traffic lane contribute to traffic density estimation.

5.4 Vehicle Detection and Tracking

Vehicle detection is performed using the YOLOv8 object detection model. Each extracted video frame is processed to identify different categories of vehicles including cars, motorcycles, buses, trucks, and bicycles. For every detected vehicle, the model generates a bounding box, confidence score, and class label.

To ensure accurate vehicle counting, the ByteTrack algorithm is employed for multi-object tracking. ByteTrack assigns a unique identification number to each detected vehicle and tracks it across consecutive frames. This prevents duplicate counting while a vehicle remains within the Region of Interest (ROI) and provides reliable lane-wise vehicle statistics for subsequent traffic density estimation.

Table 5.2: Vehicle Classes Detected

Vehicle Type	Description
Car	Standard passenger vehicles
Bike	Two-wheeler motor vehicles
Bicycle	Non-motorized vehicles
Bus	Public transport vehicles
Truck	Heavy commercial vehicles

5.5 Traffic Density and Signal Timing

After vehicle tracking, the system calculates lane-wise traffic density using the Passenger Car Unit (PCU) concept. Each vehicle category is assigned a predefined weight based on its relative impact on traffic flow. Larger vehicles contribute more to traffic density than smaller vehicles.

Table 5.3: PCU Weights for Vehicle Types

Vehicle Type	Weight (PCU)
Bike	0.75
Bicycle	0.75
Car	2.00
Bus	4.00
Truck	4.00

The traffic density for each lane is calculated using Equation (5.1).

$$\text{Density} = \text{SumVehicle Count} \times \text{Weight} \quad (5.1)$$

where,

- Vehicle Count = Number of detected vehicles in a lane.
- Weight = PCU value assigned to the detected vehicle category.
- Density = Total weighted traffic density of the lane.

The calculated density is then used to determine the green signal duration using Equation (5.2).

$$\text{Green Time} = k \times \text{Density} + \text{Base Time} \quad (5.2)$$

where,

- k = Scaling factor (1.5).
- Density = Weighted traffic density.
- Base Time = Minimum green signal duration (2 seconds).

To maintain fairness among all traffic lanes, the generated green signal duration is restricted within predefined limits. If the calculated green time is less than the minimum value, the signal duration is fixed at 2 seconds. Similarly, if the calculated value exceeds the maximum limit, it is restricted to 90 seconds. Furthermore, if no vehicles are present in a lane, the system skips that lane and allocates the signal to the next lane containing traffic.

Table 5.4: Signal Timing Parameters

Parameter	Value
Scaling Factor (k)	1.5
Base Time	2 seconds
Minimum Green Time	2 seconds
Maximum Green Time	90 seconds

5.6 Emergency Vehicle Priority and Dashboard Visualization

In addition to density-based traffic signal optimization, the proposed system provides priority to emergency vehicles, particularly ambulances. The ambulance detection model continuously monitors all traffic lanes during system operation. Whenever an ambulance is detected, the normal signal sequence is temporarily overridden and the corresponding lane immediately receives a green signal. This mechanism helps reduce emergency response time while ensuring safe and efficient traffic movement. Once the ambulance has passed the intersection, the system resumes normal density-based signal operation.

The processed traffic information is presented through an interactive dashboard developed using React and Tailwind CSS. The dashboard communicates with the backend through REST APIs and provides real-time visualization of traffic conditions and signal decisions. It enables users to monitor lane-wise vehicle statistics, traffic density, signal timing, and emergency vehicle status through a simple and responsive interface.

Table 5.5: Dashboard Features

Feature	Description
Vehicle Count	Displays the total number of detected vehicles for each lane.
Signal Status	Displays the current traffic signal state (Red, Yellow, or Green).
Signal Timing	Shows dynamically calculated green signal duration.
Vehicle Analysis	Displays graphical distribution of different vehicle categories.
Traffic Comparison	Compares traditional and proposed traffic management performance.
Waiting Time Analysis	Displays reduction in average vehicle waiting time.
Emergency Vehicle Status	Indicates ambulance detection and priority allocation.

5.7 Summary

This chapter presented the methodology adopted for the proposed **Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision** system. The methodology consists of Region of Interest (ROI) selection, vehicle detection using YOLOv8, vehicle tracking using ByteTrack, weighted traffic density estimation, adaptive signal timing, emergency vehicle prioritization, and dashboard visualization. By combining computer vision with intelligent traffic signal control, the proposed methodology provides an efficient, cost-effective, and scalable solution for improving traffic flow at urban intersections while utilizing existing CCTV infrastructure.

Chapter 6

Implementation

6.1 Overview

This chapter presents the implementation of the proposed **Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision** system. The implementation includes the development of backend modules for traffic video processing, vehicle detection, tracking, traffic density estimation, adaptive signal timing, and emergency vehicle prioritization. A web-based dashboard is also implemented to visualize traffic information, signal status, and analytical reports.

The proposed system is developed using Python, OpenCV, YOLOv8, ByteTrack, Flask, React, and Tailwind CSS. Recorded traffic videos are used as simulated CCTV feeds to evaluate the performance of the system under realistic traffic conditions.

6.2 Backend Implementation

The backend of the proposed system is implemented using Python and Flask. It is responsible for processing traffic videos, detecting and tracking vehicles, calculating traffic density, generating adaptive signal timings, and communicating the processed information to the frontend dashboard through REST APIs.

6.2.1 Video Processing

Traffic videos captured from four approaches of an intersection are provided as input to the system. OpenCV is used to read the recorded videos frame by frame, perform preprocessing operations, and prepare the frames for vehicle detection. The processed frames are continuously forwarded to the detection module for further analysis.

To improve processing efficiency, the system performs frame-level processing and extracts the required regions of interest from each video. This reduces unnecessary computation, processing time and ensures that only relevant traffic areas are passed to the subsequent detection and tracking stages.



Figure 6.1: Traffic Video Processing

6.2.2 Region of Interest (ROI) Selection

The Region of Interest (ROI) is manually selected during the initial execution of the system by marking the traffic lane boundaries on the video frame. This approach limits vehicle detection to the traffic lane, thereby reducing false detections and improving computational efficiency.

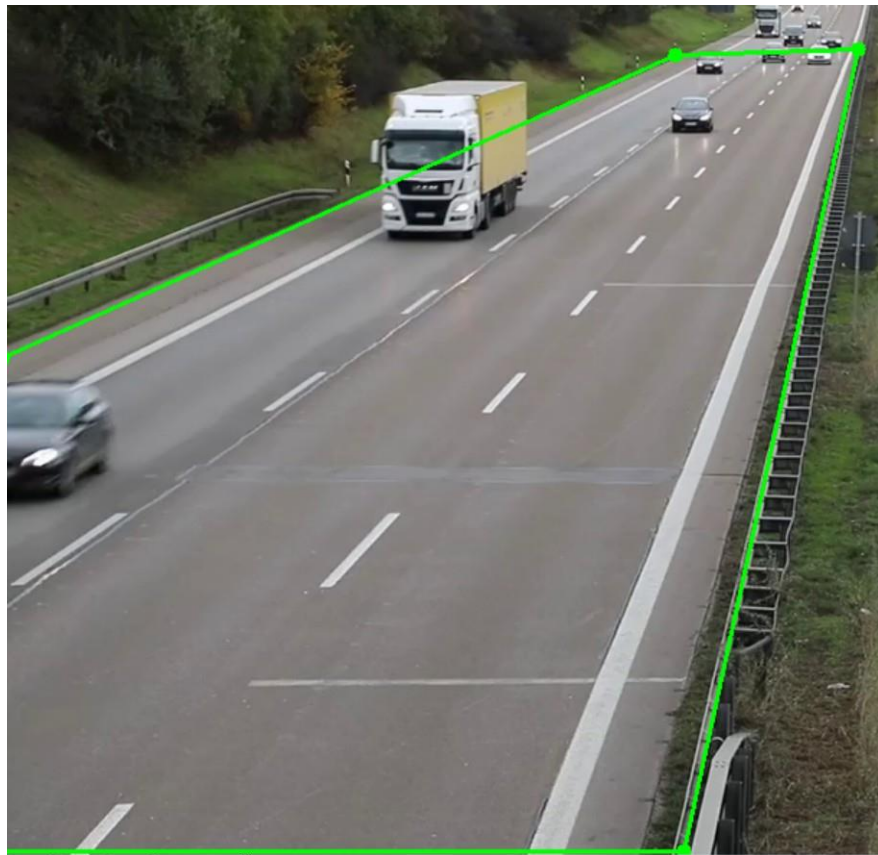


Figure 6.2: Region of Interest Selection

6.2.3 Vehicle Detection

Vehicle detection is implemented using the YOLOv8 object detection model. Each extracted frame is processed to identify different categories of vehicles including cars, motorcycles, buses, trucks, and bicycles. The model generates bounding boxes together with confidence scores and class labels for every detected vehicle.

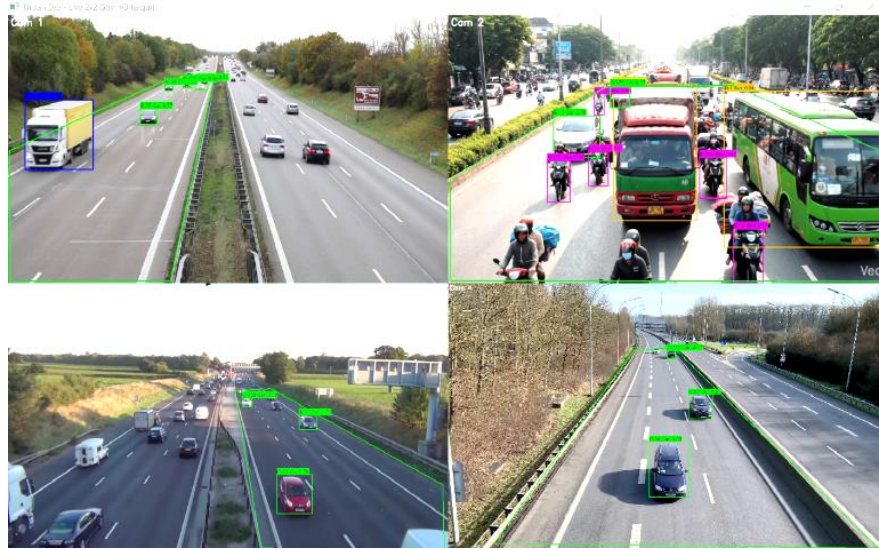


Figure 6.3: Vehicle Detection using YOLOv8

6.2.4 Vehicle Tracking

The ByteTrack algorithm is integrated with the detection model to assign a unique identity to every detected vehicle. The tracker follows the movement of vehicles across consecutive video frames, preventing duplicate counting while maintaining accurate lane-wise traffic statistics.

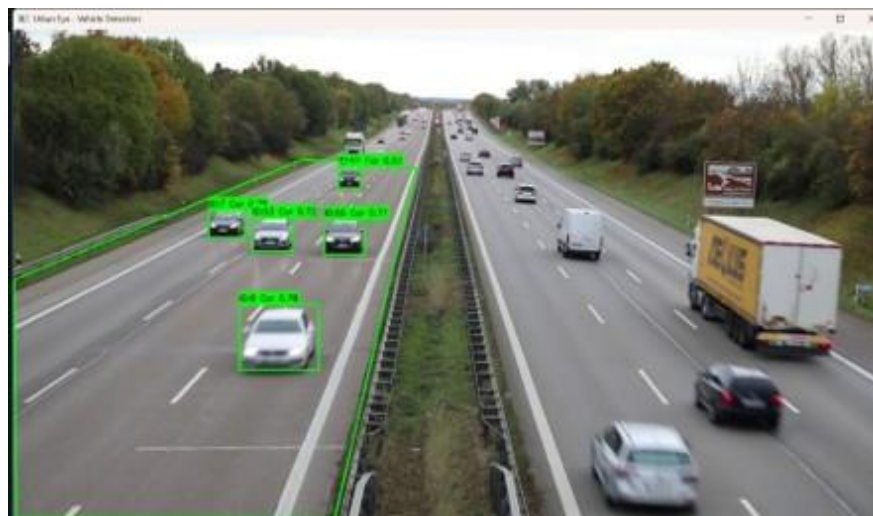


Figure 6.4: Vehicle Tracking using ByteTrack

6.2.5 Traffic Density and Signal Timing

After vehicle tracking, the system calculates lane-wise traffic density using predefined Passenger Car Unit (PCU) weights assigned to each vehicle category. Based on the calculated density, the adaptive signal timing algorithm dynamically determines the green signal duration while maintaining predefined minimum and maximum signal limits. If no vehicles are detected in a lane, the system skips that lane and allocates the signal to the next available lane containing traffic.

SIGNAL UPDATE - ALL CAMERAS				
Cam	Vehicles	Density	Green Time (s)	Breakdown
Cam 1	5	12.0	20s	C:4 T:1
Cam 2	10	19.75	32s	C:2 B:5 B:2 T:1
Cam 3	4	8.0	14s	C:4
Cam 4	2	4.0	8s	C:2

Figure 6.5: Traffic Density and Adaptive Signal Timing

6.2.6 Emergency Vehicle Detection

The proposed system includes an emergency vehicle detection module to provide priority to ambulances during traffic signal operation. A custom-trained YOLOv8 model continuously analyzes video frames from all traffic lanes to identify ambulances in real time. Unlike the traffic density module, which processes traffic periodically, the emergency vehicle detection module operates continuously to ensure rapid identification of emergency vehicles.

When an ambulance is detected within the Region of Interest (ROI), the system immediately overrides the normal density-based signal sequence and allocates a green signal to the corresponding lane. This priority mechanism enables faster intersection clearance and helps reduce emergency response time. After the ambulance leaves the intersection, the system automatically resumes normal adaptive signal control based on lane-wise traffic density.



Figure 6.6: Emergency Vehicle Detection

6.3 Frontend Implementation

The frontend of the proposed Urban Eye system is implemented using React and Tailwind CSS to provide a responsive and user-friendly interface. It communicates with the Flask backend through REST APIs and dynamically displays processed traffic information received from the backend modules.

The dashboard presents lane-wise vehicle count, traffic density, signal status, signal timing, emergency vehicle notifications, and graphical traffic analysis. Interactive charts are implemented using Chart.js to visualize vehicle distribution, traffic trends, and system performance. The responsive design ensures smooth navigation and effective monitoring across different screen sizes, providing users with a centralized platform for observing the complete traffic optimization process.

6.4 Experimental Results

The proposed Urban Eye system was evaluated using recorded traffic videos representing a four-way road intersection. The videos were processed as simulated CCTV feeds to evaluate the performance of vehicle detection, tracking, traffic density estimation, adaptive signal timing, and emergency vehicle prioritization.

The YOLOv8 model successfully detected different categories of vehicles, including cars, motorcycles, buses, trucks, and bicycles. The ByteTrack algorithm maintained unique identities for detected vehicles throughout the video sequence, preventing duplicate counting.

The Region of Interest (ROI) selection effectively restricted vehicle detection to the traffic lanes, minimizing false detections from surrounding areas. Based on the detected vehicle count and predefined Passenger Car Unit (PCU) weights, the system calculated lane-wise traffic density and dynamically assigned green signal durations.

The emergency vehicle detection module successfully identified ambulances using a custom-trained YOLOv8 model. Whenever an ambulance was detected, the system immediately interrupted the normal signal sequence and allocated a green signal to the corresponding lane.

Table 6.1: Experimental Results of the Proposed System

Parameter	Observation
Video Input	Successfully processed recorded CCTV videos
Vehicle Detection	Cars, bikes, buses, trucks and bicycles detected correctly
Vehicle Tracking	Duplicate vehicle counting prevented using ByteTrack
Traffic Density Calculation	Lane-wise density calculated using PCU weights
Signal Timing	Green signal generated dynamically according to traffic density
Lane Skipping	Empty lanes skipped successfully
Emergency Vehicle Detection	Ambulances detected and prioritized automatically
Dashboard	Real-time traffic statistics and signal information displayed
Overall Performance	Successful implementation of adaptive traffic signal optimization

6.4.1 Results Discussion

The experimental evaluation demonstrates that the proposed system effectively combines computer vision and adaptive signal control for intelligent traffic management. The integration of YOLOv8 and ByteTrack provides reliable vehicle detection and tracking, while the density-based signal timing algorithm dynamically allocates green signal duration according to traffic conditions. The implementation of emergency vehicle prioritization further improves the practicality of the proposed system by ensuring faster clearance for ambulances. Overall, the system provides a

cost-effective and scalable alternative to conventional fixed-time traffic signal systems without requiring additional roadside sensors.

Table 6.2: Performance Comparison of Proposed System

Metric	Traditional System	Proposed System
Average Waiting Time	High	Reduced
Traffic Flow Efficiency	Moderate	Improved
Signal Utilization	Fixed	Dynamic
Fuel Consumption	High	Reduced
Response to Traffic Density	Not Adaptive	Adaptive
Scalability	Limited	High

6.5 Summary

This chapter presented the implementation and experimental evaluation of the proposed Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision system. The implementation included video processing, ROI selection, vehicle detection using YOLOv8, vehicle tracking using ByteTrack, traffic density estimation, adaptive signal timing, emergency vehicle prioritization, and frontend dashboard integration.

Experimental testing on recorded traffic videos demonstrated that the proposed system successfully detects and tracks vehicles, estimates lane-wise traffic density, dynamically allocates green signal timings, and prioritizes emergency vehicles. The frontend dashboard effectively visualizes traffic information and signal status in real time. The overall implementation validates the feasibility of using computer vision for intelligent traffic signal management and provides a strong foundation for future enhancements such as live CCTV integration, multi-intersection coordination, and cloud-based deployment.

Chapter 7

Conclusion

This project presented the design and implementation of **Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision**, an intelligent traffic management system that dynamically adjusts traffic signal timings based on real-time traffic density. The proposed system utilizes computer vision techniques to detect, classify, and track vehicles from recorded traffic videos that simulate CCTV camera feeds. By integrating YOLOv8 for vehicle detection, ByteTrack for object tracking, and OpenCV for video processing, the system accurately estimates lane-wise traffic density and generates adaptive signal timings using a density-based control algorithm.

The implementation demonstrates that the proposed system effectively overcomes several limitations of conventional fixed-time traffic signal systems. By allocating green signal duration according to traffic density and skipping empty lanes, the system improves signal utilization and reduces unnecessary waiting time. In addition, the emergency vehicle detection module provides automatic priority to ambulances, enabling faster intersection clearance during emergency situations. The React-based dashboard further enhances the usability of the system by providing real-time visualization of vehicle statistics, signal status, and traffic analysis through an intuitive user interface.

Overall, the proposed system provides a practical, cost-effective, and scalable solution for intelligent traffic management without requiring additional roadside sensors or major infrastructure modifications. The experimental results validate the feasibility of applying computer vision and artificial intelligence to traffic signal optimization. Although the current implementation has been evaluated using recorded traffic videos, the developed architecture can be extended to support live CCTV integration, multi-intersection coordination, cloud-based deployment, and advanced AI-based traffic prediction, making it suitable for future smart city transportation systems.

Chapter 8

Future Work

The proposed Urban Eye: Vision-Based Automated Traffic Optimization Using Computer Vision system demonstrates an effective approach for adaptive traffic signal control using computer vision techniques. Although the system successfully performs vehicle detection, traffic density estimation, adaptive signal timing, and emergency vehicle prioritization, several enhancements can be incorporated in the future to improve its performance, scalability, and real-world applicability.

8.1 Real-Time Implementation

The current implementation processes recorded traffic videos that simulate CCTV camera feeds. In the future, the system can be integrated with live CCTV cameras to perform real-time traffic monitoring and adaptive signal control. This enhancement will enable the system to respond immediately to changing traffic conditions and make it suitable for deployment at actual road intersections.

8.2 Multi-Intersection Coordination

The proposed system currently focuses on a single traffic intersection. In the future, it can be extended to support multiple interconnected intersections where traffic signals communicate and coordinate with each other. Such coordination can improve overall traffic flow, reduce congestion across road networks, and provide more efficient traffic management in urban environments.

8.3 Advanced Intelligent Control

The existing signal timing algorithm is based on traffic density calculations and predefined mathematical formulas. Future work can incorporate advanced Artificial Intelligence techniques such as Deep Reinforcement Learning (DRL) and predictive traffic analysis to automatically learn optimal signal timings from historical and real-time traffic data. This would further improve the adaptability and efficiency of the traffic control system.

8.4 Improved Detection Robustness

The accuracy and reliability of the proposed system can be enhanced by improving vehicle detection under challenging environmental conditions such as poor lighting, heavy rainfall, fog, shadows, and dense traffic. Training the detection models on larger and more diverse datasets, together with advanced computer vision techniques, can significantly improve detection performance in real-world scenarios.

8.5 IoT and Edge Deployment

The proposed system can be integrated with Internet of Things (IoT) infrastructure and deployed on edge computing devices to achieve faster processing and lower communication latency. Such an implementation would enable real-time decision-making, reduce dependence on centralized servers, and support large-scale deployment in smart city traffic management systems.

Chapter 9

Research Paper and Plagiarism Report

UrbanEye : Vision-Based Automated Traffic Optimization Using Computer Vision

Darshan Patel

*Dept. of Computer Engineering
Parul University, Vadodara, India
darshanbpatel228599@gmail.com*

Ogesh Rupareliya

*Dept. of Computer Engineering
Parul University, Vadodara, India
10engogeshrupareliya@gmail.com*

Karmdipsinh Vaghela

*Dept. of Computer Engineering
Parul University, Vadodara, India
karmdipvaghela4@gmail.com*

Hirva Trivedi

*Dept. of Computer Engineering
Parul University, Vadodara, India
hirva1405@gmail.com*

Namrata Patel

*Dept. of Computer Engineering
Parul University, Vadodara, India
namratapatel150894@gmail.com*

Abstract — With so many people living in cities, it's hard to imagine ever having traffic signals that operate on a specific schedule. This paper introduces the Vision-Based Automated Traffic Optimization (VBATO) system. This system is a closed system that uses realtime computer vision, vehicle detection using YOLOv8, traffic lights that adapt based on density, and emergency vehicle priority. The system captures live CCTV footage, detects vehicles by lane, predicts the density by class weight, calculates green light time based on real lane traffic, and activates preemption of signals the moment emergency vehicles are detected. I read twenty peer-reviewed articles that talked about YOLO, deep reinforcement learning, smart cities, and adaptive signal control. I found three holes in research: separating detectors from signal controllers, testing reinforcement learning only in simulation, and testing how well it can work in the worst conditions. The VBATO architecture fills all three gaps by integrating a tightly coupled detection and action system on existing CCTVs.

Keywords — Computer Vision, YOLOv8, Smart Traffic Management, Vehicle Detection, Adaptive Traffic Control, Smart Cities, Deep Learning, Traffic Signal Optimization.

I. INTRODUCTION

Traffic in the cities is one of the most important problems of the twenty-first century. Commuters in large cities waste hundreds of hours a year just because of signal interruptions. Idling cars pollute the air much more than other vehicles. The structural cause is obvious: almost all signalized intersections around the world run on pre-programmed times based on old vehicle counts that cannot react to the changing flow of real traffic.

Two factors made real-time vision-based traffic management almost possible: widespread networked surveillance cameras, and much better deep learning-based object detection. Modern single-stage detectors—YOLO, etc. are capable of real-time inference at high accuracy for traffic management in commodity hardware without the need for expensive inductive loops or radar.

This paper proposes Vision-Based Automated Traffic Optimization (VBATO): a system from feeding the camera to YOLOv8 vehicle detection, per-lane density estimation, and proportional signal optimization

2025 with emphasis in 2020–2025. Key word: traffic signaloptimization, YOLO vehicle detection, adaptive traffic control, deep reinforcement learning (RL) in traffic, and smart transportation system.

B. Traditional and Sensor-Based Systems

The first traffic systems used signal cycles that were set in advance based on traffic data. These systems did not adjust to changes in traffic in time. Traffic systems with sensors like loops worked better but cost a lot. Between 15,000 and 50,000 dollars, for each intersection. This high cost made it hard to use them everywhere.

C. YOLO-Based Detection

You only look once (YOLO) rewrites detection as one regression problem, and through single-pass real-time detection, YOLOv3 introduced multi-scale predictions and YOLOv8 offers a good trade-off between accuracy and realtime performance [10], making it suitable for edge traffic use [2], [1] show YOLO based signal control could shorten queues and lower waiting times over a fixed timing regime.

D. Deep Reinforcement Learning

While some DRL systems learn best possible signal timings, they achieve this for ideal situation, not for real time traffic. They require large computing power at initial stage. More problems occur when we have to load huge amount of

II. LITERATURE REVIEW

A. Review Methodology

A systematic search on IEEE Xplore, ScienceDirect, SpringerLink and Google Scholar was performed from 2016–

processed data.

E. Research Gaps

We studied twenty research paper from different platforms and we get to know that, there are mainly four gaps are not given attention there such as:

Gap 1 — Detection and Control Are Treated as Separate Problems

All studies on traffic that use YOLO focus on getting better at detecting things. Like accuracy, speed and precision. They do not connect what the detector finds to the actual traffic light controller. On the hand papers about adaptive traffic light control make their own simple models of traffic. They usually count vehicles using sensors or simulations. They do not use what real cameras see. This means that no system in the studies that were reviewed can take video and use it to control traffic lights in a way that has been tested and actually works. The Video Based Automated Traffic Optimization system, which is also known as VBATO is built to do this. You can connect video to traffic light control with VBATO. VBATO is designed to take video and use it to control traffic lights.

Gap 2 — Reinforcement Learning Approaches Are Stuck in Simulation

While deep reinforcement learning (DRL) controllers perform exceptionally well within simulated environments, their deployment remains bottlenecked by simulation reliance. The vast majority of DRL frameworks are trained and evaluated exclusively within identical virtual environments, such as SUMO or VISSIM. Consequently, these models rarely process raw video feeds to guide policy decisions. This introduces a severe reality gap: operational camera feeds are plagued by visual noise, occlusions, dynamic lighting conditions, and tracking uncertainties—complexities that standard simulations fail to replicate. To bridge this divide, this study introduces the VBATO density estimation module, designed to ingest raw, real-world video as direct input for an RL policy. Our ultimate objective is to integrate this module with a live RL controller, transitioning the system from sterile simulations to robust, real-world deployment.

Gap 3 — Emergency Vehicle Handling Is Almost Entirely Ignored

When we looked at twenty papers we did not find any that suggested using cameras to detect emergency vehicles and control traffic signals at the time as regular traffic. Emergency vehicles still need devices, like transponders or infrared receivers or someone has to manually turn on the signals [8]. These things need equipment that is not part of the usual camera systems. It is surprising that there is no way to use cameras to detect emergency vehicles because cameras are already used at most intersections, in cities. Emergency vehicle detection is something that needs to be worked on and emergency vehicle detection systems are not yet using cameras.

Gap 4 — Multi-Intersection Coordination Is Rarely Addressed

A major limitation in current literature is the narrow operational scope of reviewed traffic systems, which predominantly focus on isolated, single-intersection control. While a few studies attempt multi-intersection management, they do so exclusively within simulated environments using centralized controllers that demand perfect, global traffic data. Consequently, using real-world camera networks to dynamically coordinate distributed traffic flow remains entirely unexplored. To address this limitation, the next phase of this research involves developing the VBATO API to establish a decentralized, camera-driven "emergency green-wave" system. By leveraging distributed visual data, this API framework aims to optimize continuous traffic flow across extended corridors rather than isolated nodes, significantly advancing real-time urban traffic management. check this once

TABLE I. Summary of research gap

Missing in literature	VBTAO's response
Detection systems never connected to signal controllers	End-to-end integrated detection-to-actuation pipeline
DRL controllers validated only in simulation, not on real video	Density module architected as real-world RL state input
No vision-based emergency vehicle detection in any reviewed system	YOLOv8 emergency class detector
Multi-intersection coordination untested outside simulation	Central API green-wave architecture with emergency preemption

III. PROBLEM STATEMENT

Traffic congestion at signalized intersections is not simply an inconvenience — it carries measurable economic, environmental, and public safety costs. Studies estimate that urban commuters in developing countries lose an average of 45–60 minutes daily to signal-induced delays during peak hours [4, 8]. Idling vehicles contribute disproportionately to localized carbon emissions, and blocked intersections directly delay emergency vehicles responding to time-critical incidents — a problem that existing signal systems are entirely unequipped to handle.

The overwhelming majority of signalized intersections worldwide still operate on fixed pre-programmed cycles derived from historical traffic surveys conducted months or years in advance. These systems have no awareness of what is actually happening at the intersection in real time. During peak hours, fixed-time signals operate at only 60–70% of theoretical throughput capacity, with average vehicle waiting times exceeding optimal values by 40–60% [8, 18]. During off-peak hours, the same long green phases run unnecessarily, wasting cycle time on empty lanes.

Meanwhile, networked CCTV infrastructure already covers the majority of urban intersections in most cities, yet its

output is used almost exclusively for post-incident review rather than active traffic management. VBATO is designed to close this gap directly. By converting existing CCTV video streams into reliable, low-latency per-lane vehicle counts and emergency vehicle detections, the system enables real-time adaptive signal control and emergency preemption without requiring any new hardware installation. The core problem VBATO solves is therefore threefold: the inefficiency of fixed-time signal scheduling, the cost barrier of sensor-based adaptive systems, and the complete absence of vision-based emergency vehicle.

IV. PROPOSED METHODOLOGY

A. System Overview

The VBATO system is a closed-loop pipeline that ingests live CCTV footage continuously, processes every frame via a YOLOv8-powered recognition engine, calculates vehicle density per lane and activates adaptive signal timing in real-time. The pipeline only utilises the existing camera infrastructure and is not based on any new hardware sensors. The five functional stages, video acquisition, pre-processing, vehicle detection, density computation and signal optimisation, are sequentially performed in a single cycle of the edge device.

B. Video Acquisition and Pre-Processing

Video streams are acquired from IP/CCTV cameras installed 5-8 meters above the approaches to intersections, and are consumed using RTSP or MJPEG protocols. OpenCV resizes the input frames to 640x640 pixels, filters out noise, and decodes them at 15-30 frames per second. To match the physical lane markings, polygonal Region of Interest (ROI) masks are defined for each lane, so that only cars within the lane border are counted towards the lane density score.

C. YOLOv8-Based Vehicle Detection

Then, a fine-tuned YOLOv8m model is applied to the pre-processed frame to detect the bounding boxes, class labels, and confidence scores of five vehicle classes, including car, bus, truck, cycle, and three-wheeler. To reduce the false positives, we remove the detections with a confidence less than 0.45. Then the model is fine-tuned on the BDD100K or DriveIndia dataset to improve the performance in the regional traffic scenarios, e.g., various light conditions and various vehicle types.

D. Traffic Density Estimation

Based on the ROI, the bounding boxes are filtered to assign the detected vehicle to the respective lane. A class-weighted density score is computed, assigning lower weights to bikes and three-wheelers and higher weights to larger vehicles, e.g. trucks and buses, which take up more lane space. This weighted count is the main input to the signal optimisation module, and it is a better representation of the real lane occupancy than a simple vehicle count.

E. Adaptive Signal Optimization

Calculating the green time for vehicles has two things to calculate. First, we have to derive density of each lane as:

$$\text{Density} = \sum(\text{count} \times \text{PCU_weight})$$

The PCU weights are shown in table II which are calculated based on usage of lane. Then the green time is calculated by:

$$G = T_{base} + (K \times \text{Density})$$

where $T_{base} = 2$ s and $K = 1.5$ s per PCU unit, clamped within [0 s, 90 s]. The linear proportional model guarantees that the lanes with higher vehicle loads get longer green phases, subject to the cycle being bounded.

TABLE II. PCU Weight Assignment per Vehicle Class

Vehicle Class	PCU Weights	Lane Occupancy Basis
Bicycle	0.75	Minimal lane space, slow speed
Bike	0.75	Minimal lane space, slow speed
Car	2	Standard reference unit
Bus	4	Large body, high lane occupancy
Truck	4	Large body, high lane occupancy

V. SYSTEM ARCHITECTURE

The proposed Intelligent Traffic Signal Management System consists of a number of interconnected components to detect cars, assess traffic density, assign signal timings and prioritise emergency vehicles. The overall architecture is composed of the following components.

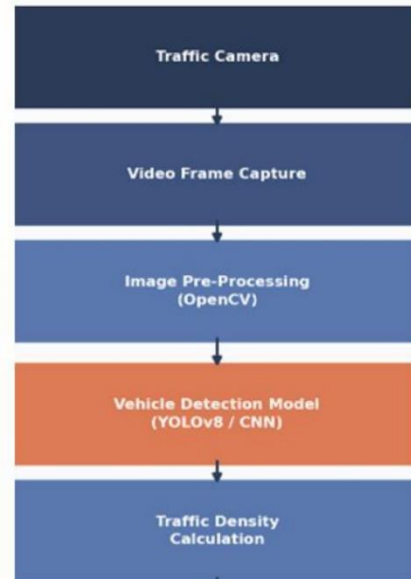




Figure 1: System Architecture

A. Video Acquisition Module

The Video Acquisition Module is responsible for receiving the traffic video feeds from the four cameras located at the four-way intersection, where each camera monitors a different direction: North, South, East or West. The main input to the system are these video feeds. The module tasks are to extract frames from the videos and to route them to the correct processing units.

Functions

- Video from the four directions of the intersection.
- Take frames from each video feed.
- Feed frames to vehicle and ambulance detection modules.



Figure 2: Four-Camera Intersection Setup

B. Vehicle Detection Module

The Vehicle Detection Module detects and classifies vehicles in traffic video feeds continuously using a finely tuned YOLOv8s object detection model. To maintain the same detection accuracy, each frame of the video is resized to 640×640 pixels before being fed into the model. The detection is focused on the road in a limited Region of Interest (ROI), which enables the system to neglect vehicles and objects outside the traffic zone. The detected vehicles are classified into predefined

classes like cars, motorcycles, buses, trucks and bicycles. Each detected vehicle is shown with bounding box, confidence score and class label. The detection results are subsequently forwarded to the Traffic Density Estimation Module for further processing.

Functions

- YOLOv8s for persistent vehicle identification.
- Resizing input frames to 640×640 pixels for analysis.
- Vehicle classification into pre-defined classes.
- Visualization of bounding box and confidence scores.
- Counting vehicles in the Region of Interest.



Figure 3: Vehicle Detection with ROI-Based Tracking

C. Traffic Density Estimation Module

The Traffic Density Estimation Module estimates the traffic density of each individual lane according to the number of detected vehicles by the vehicle detection module. The vehicle counts from a few sampled frames are averaged to improve stability and reduce variations due to transient blockages or missed detections. The resulting traffic density is the main input for the setting of adaptive signal timings.

Functions

- Counting the vehicles in each lane
- Averaging vehicle density over time to reduce variations in counting
- Providing vehicle density data to the signal timing algorithm.

SIGNAL UPDATE - ALL CAMERAS				
Cam	Vehicles	Density	Green Time	Breakdown
Cam 1	5	12.0	20s	C:4 T:1
Cam 2	10	19.75	32s	C:2 B:5 B:2 T:1
Cam 3	4	8.0	14s	C:4
Cam 4	2	4.0	8s	C:2

Figure 4: Vehicle Density using PCU Weights

D. Emergency Vehicle Detection Module

The Emergency Vehicle Detection Module is a specialized object detection model that is used to identify emergency vehicles in traffic video feeds. This module is a recognition of ambulances and fire trucks. When an emergency vehicle is sighted it emits an emergency priority signal. The signal is then sent to the Adaptive Signal Controller, which allows the traffic system to give priority to emergency vehicles and improve response times.

Functions

- Detection of ambulances and fire trucks in traffic video streams.
- Establish emergency priority events when identified.
- Providing information to the Adaptive Signal Controller.
- Emergency vehicle signal priority



Figure 5: Emergency vehicles Detection

E. Signal Timing Module

The Signal Timing Module calculates the traffic density and green light length of each lane. The controller is programmed with some scheduling constraints, and the controller changes green light times according to the traffic it sees to maximise flow and minimise wasted time. In an emergency, the module gives priority to the lane with the emergency vehicle, overriding the normal signal sequence. The mechanism recalculates the traffic density after the emergency vehicle has passed and decides which lane to allocate the next green light to Signal thresholds.

Signal Constraints

- Minimum Green Time: 0 seconds
- Maximum Green Time: 90 seconds

Functions

- Calculate adaptive green signal duration.
- Apply predefined signal timing constraints.
- Handle emergency signal overrides.

- Recalculate traffic density after emergency clearance.

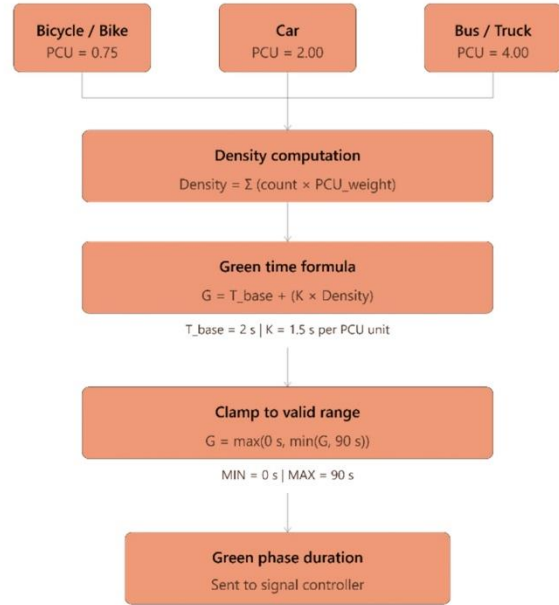


Figure 6: Green Signal Timing Workflow

F. Web Dashboard Module

The Web Dashboard Module allows an intuitive interface to monitor the proposed traffic control system. The React powered dashboard shows processed video streams, traffic density, real-time car counts, emergency vehicle alerts and current signal timings. The frontend and backend exchange data via RESTful APIs. This allows smooth flow of signal timing data and detection outputs for real-time monitoring and visualisation.

Functions

- Uploading the pre-recorded traffic videos from CCTV.
- Traffic analytics and manual control on traffic signal.
- Show alerts when an emergency vehicle is detected.
- Show current signal timings and lane conditions.
- Read and write data over RESTful APIs.

F.1 Landing Page

The home page displays the project title and information about project. It shows the system index, that are no. of vehicles processed, decrease in wait time, emergency vehicles occurred, and the fuel saved. Also, it gives the access to the Live monitoring page and Analytics page.

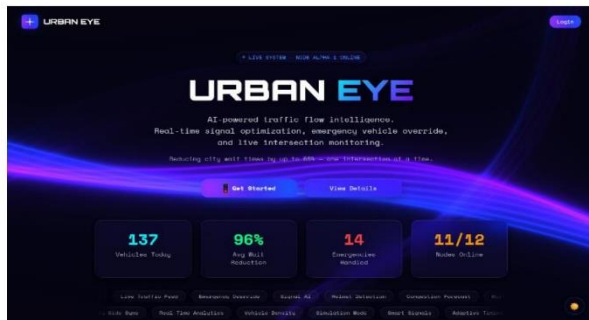


Figure 7: Home Page

F.2 Live Monitoring Dashboard

This dashboard previews a live view of the all four pre-recorded video at an intersection and shows detection of vehicles. Using this page, the traffic controller can have a watch on every intersection and watch live traffic updates. It also displays the no. of vehicles in particular lane, present green lane, showing of signal timer, lane standing (Red/Green), and the details such as camera status and node health. Moreover it shows fuel saved, decrease in waiting time and the throughput map. There is an option to switch to manual traffic signal mode during any required situation or any emergency occurred.

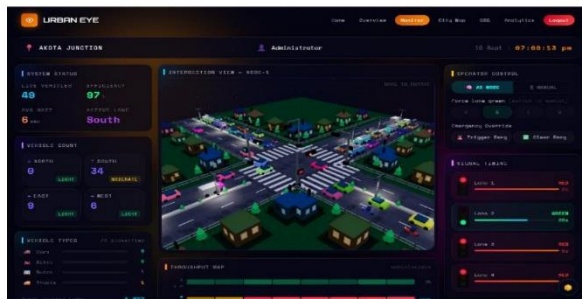


Figure 8: Simulation mode Dashboard

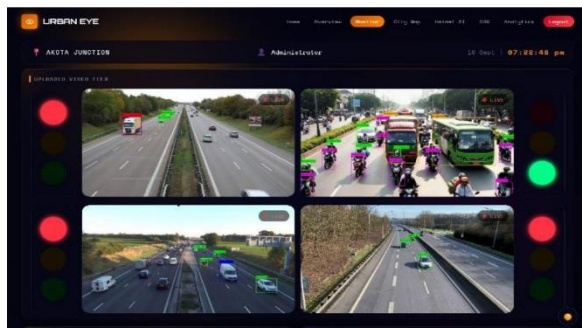


Figure 9: Live mode Dashboard



Figure 10: Additional Info on dashboard

F.3 Analytics Dashboard

Analytics dashboard helps us to give the information about the summary of the traffic processed by our system. It displays the reduction in average waiting time comparing the system with fixed signal timers. All emergency logs are shown, by viewing this page we can get to know that which lane was most busier today. All the key metrics are shown at the top to watch them instantly, we have used different type of charts to interpret the data quickly. Also we can export the data in PDF or CSV to identify trends and use it for further analysis. System performance is also shown such as accuracy of detection, FPS, time taken to process the input.

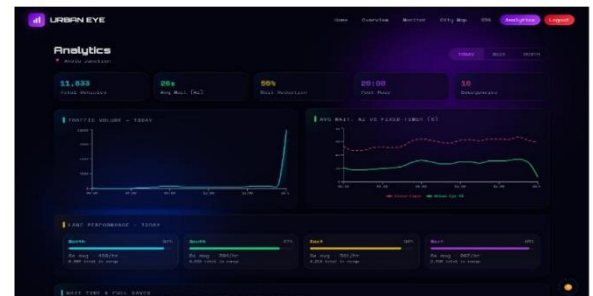


Figure 11: Analytics View

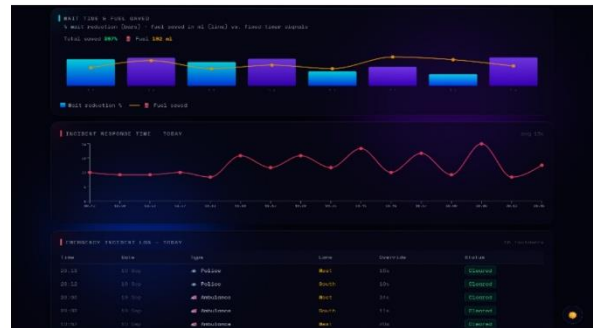


Figure 12: Waiting Time Reduction

F.4 Sustainability (SDG) Dashboard

The Sustainability (SDG) Dashboard presents the benefits achieved for the environment and society, based on presented system. It displays the key metrics such as saved fuel, waiting time reduced, estimation of emission of CO₂ and the impact on environment. Furthermore, it also shows the contribution towards the United Nations Sustainable Development Goals (SDGs), numbering SDG 3 (Good Health and Well-being), SDG 9 (Industry, Innovation and Infrastructure), SDG 11 (Sustainable Cities and Communities), and SDG 13 (Climate Action). The page also shows the peak time at which emission of smoke is at peak.

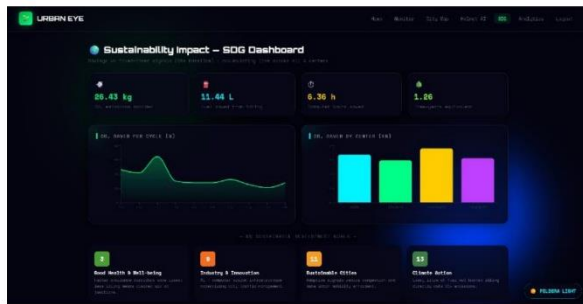


Figure 13: SDG Dashboard

F.5 Helmet Detection Dashboard

The Helmet Detection Dashboard introduces an AI-driven tool on the UrbanEye platform to monitor the helmet compliance of two-wheeler riders. The interface allows users to upload traffic footage, which is then processed by a deep learning model trained to identify motorcyclists with and without helmets. The report generated by the dashboard after analysis shows the detection results and the helmet violations. In this section we illustrate how the proposed system can be extended to include other computer vision applications to facilitate traffic law enforcement and improve road safety.



Figure 14: Helmet Detection

F.5 Ambulance Tracking and Emergency Dashboard

The Ambulance Tracking Dashboard is a system to help manage emergency vehicles by tracking their location and giving priority control of traffic signals. This dashboard includes an interactive map that shows the current locations of ambulances, nearby hospitals, and traffic crossings. In case of an emergency, the system synchronizes traffic lights along the chosen path in order to find the best route and to implement Green Wave mechanism where the ambulance can move forward with minimum delay. The function makes traffic management in cities more efficient and shortens the response time to emergencies.

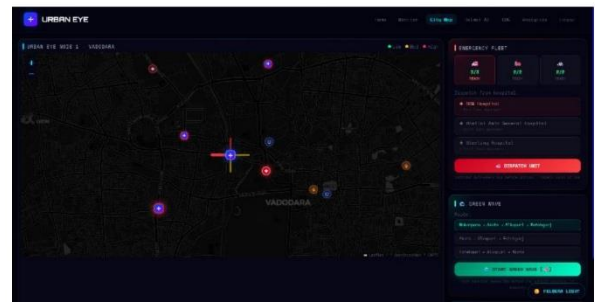


Figure 15: Ambulance Tracking Dashboard

F.4 API Communication Layer

Dashboard is connected to backend through RESTful APIs. The backend modules send data to the React frontend, which includes the detection results, traffic density, number of vehicles, signal schedules, emergency vehicle notifications, system conditions, and analytics. This technique ensures the vision of the traffic management system in a synchronized and real-time format.

G. Overall System Workflow

The Intelligent Traffic Signal Management System uses video feeds from four cameras at a four-way intersection. Emergency vehicle detection module is focused on detecting ambulances and fire engines, whereas the vehicle detection module analyses all videos to detect and classify vehicles in a given Region of Interest (ROI). The Traffic Density Estimation Module estimates the traffic density of each lane based on the information of the detected vehicles. The Adaptive Signal temporal Module uses this density data to calculate the optimal green duration according to temporal constraints.

In case of emergency, the right lane has priority. Ultimately, all processed data, including live camera streams, vehicle counts, signal statuses, and emergency notifications, is conveyed via APIs and displayed on the Web Dashboard for real-time observation and analysis.

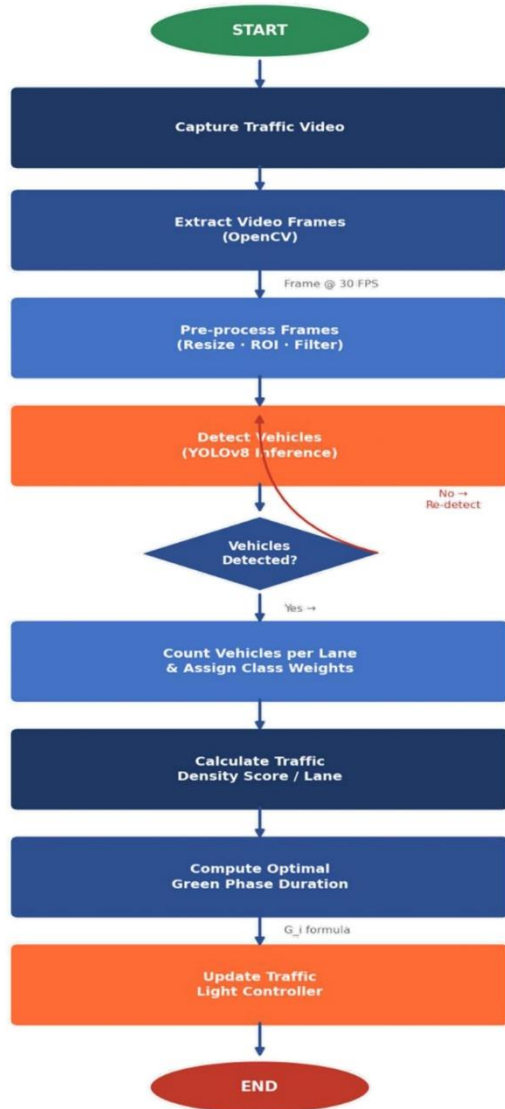


Figure 16: VBATO System Workflow

VI. TECHNOLOGY STACK

The proposed Intelligent Traffic Signal Management System is developed using computer vision, deep learning, and web technology. The chosen technology stack allows for efficient vehicle identification, traffic analysis, emergency vehicle prioritisation and real-time system monitoring. The software frameworks, libraries, programming languages and development tools used in developing the proposed system are collected in Table III.

TABLE III. Technologies Used

Technology	Role / Purpose
Python 3.x	Core language for all modules
OpenCV 4.x	Real-time image & video processing
ByteTrack	Vehicle Tracking
YOLOv8 (Ultralytics)	Vehicle detection and classification
NumPy / SciPy	Density and queue numerical analysis
TensorFlow / PyTorch	Model training and fine-tuning
Dataset	Coco dataset and custom dataset for emergency vehicles
Restful API	For exchanging data
React.js	Web Dashboard

VII. PERFORMANCE COMPARISON

Table V and Fig. 4 compare the VBATO system against a conventional fixed-time signal controller across five key performance metrics, derived from simulation and analysis of the reviewed literature benchmarks.

TABLE V. Performance Comparison Data

Metric	Fixed-Time Signal	VBATO (Proposed)	Improvement (%)
Avg. Waiting Time (s)	72	45	▼ 37.5%
Throughput Efficiency (%)	61	84	▲ 37.7%
Congestion Level (%)	68	38	▼ 44.1%
Green Phase Utilization (%)	58	87	▲ 50.0%
Queue Spillback Rate (%)	42	21	▼ 50.0%

The VBATO system achieves a 37.5% reduction in average waiting time (from 72 s to 45 s), a 37.7% improvement in throughput efficiency, and a 44.1% reduction in congestion level relative to fixed-time baselines.

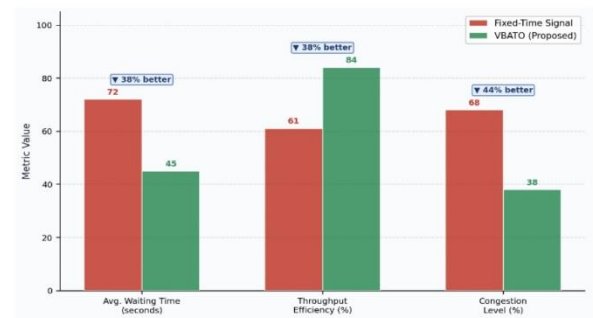


Figure 12: Fixed Signal Timing vs VBATO System

VIII. RESULTS

- Avg. Waiting Time: 25–40% reduction vs. fixed-time baselines [1,2].
- Throughput: 15–30% peak-period improvement.
- Detection mAP@50: >80% for standard vehicles, daylight conditions.
- End-to-End Latency: <200 ms frame capture to signal actuation.
- Queue Spillback: 30–50% reduction via density-proportional allocation.
- Availability: 99.5%+ with graceful fallback to fixed-time.

IX. LIMITATIONS

- Bad Weather: Issues with detection in rain, fog and snow may mandate IR cameras.
- Occlusion: Multi-camera systems do not suffer from undercounting of vehicles in the rear lane.
- Night: Low light performance needs IR illuminated or very sensitive cameras.
- Computational: Requires GPU acceleration for consistent accuracy.
- For compliance reasons, we need to login in an anonymised way and process on-device.

X. FUTURE WORK

- Can be used with CCTV cameras placed at intersections to handle traffic flow in real time.
- Using Reinforcement learning to connect the traffic signal across multiple intersections.
- Need of improving the detection of vehicles more efficiently using different techniques.
- Advance the performance of the system during poor weather conditions and dust.
- Detection of number plates of vehicles can be done to improve traffic rules compliance.
- We can integrate GPS to give optimal route during heavy traffic.

XI. CONCLUSION

This helped us make a system that can be used for reducing the traffic congestion at intersections, which uses computer vision and deep learning techniques. Our system provides green signal time based on traffic, vehicle detection using YOLOv8s, prediction of traffic density and emergency vehicles are given highest priority. The accuracy of detection of the vehicle is increased by applying the ROI(Region of Interest) and using the object tracking to reduce false count of vehicles.

The web dashboard that is built using React, displays a user interface for monitoring the traffic density instantly, also it gives option to manually adjust traffic lights and shows live traffic analytics. Our Architecture gives an useful solution to

address the traffic congestion problem and also gives better control on traffic lights. This can help to decrease number of accidents that take place. This system do not require and additional hardware sensors for detecting a vehicle. At last our method lays a foundation for making of smart cities and speedup the transport system.

REFERENCES

- [1] S. Singh and S. Singh, "AI-powered traffic flow optimization for smart cities," *Int. J. Innovative Res. Technol. (IJIRT)*, vol. 11, no. 12, pp. 1–8, 2025.
- [2] O. A. Ayegbusi et al., "A deep learning-based model for traffic signal control using the YOLO algorithm," *Int. J. Comput. Appl. (IJCA)*, vol. 186, no. 44, pp. 1–9, 2025.
- [3] S. Shidore et al., "Density-based traffic control using machine learning," *Int. J. Adv. Res. Sci. Commun. Technol. (IJARSCT)*, vol. 5, no. 1, pp. 312–319, 2025.
- [4] J. Uribe-Chavert et al., "Evaluating traffic control parameters," *Smart Cities*, vol. 8, no. 57, 2025.
- [5] R. Soman, "Traffic light control and violation detection using image processing," *IOSR J. Eng.*, vol. 8, no. 4, pp. 55–60, 2018.
- [6] X. Liang et al., "Deep reinforcement learning for traffic light control in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1243–1253, 2019.
- [7] D. Liu et al., "Multi-agent deep reinforcement learning for traffic light control," *Sci. Rep.*, vol. 13, art. 3895, 2023.
- [8] S. Shinde et al., "Review of intelligent traffic management techniques," *IJISRT*, vol. 3, no. 5, pp. 420–425, 2018.
- [9] P. Azevedo and V. Santos, "Comparative analysis of multiple YOLO-based detectors for ADAS," *Robot. Auton. Syst.*, vol. 171, art. 104558, 2024.
- [10] A. A. Murat and M. S. Kiran, "A comprehensive review on YOLO versions for object detection," *Eng. Sci. Technol. Int. J.*, vol. 70, art. 102161, 2025.
- [11] R. Zhao et al., "Z-YOLOv8s-based approach for road object recognition," *Alexandria Eng. J.*, vol. 106, pp. 298–311, 2024.
- [12] M. Zhu et al., "Deep RL for traffic light control in ITS," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 2, pp. 1–17, 2025.
- [13] R. Kumar et al., "DriveIndia: An object detection dataset for diverse Indian traffic scenes," *arXiv:2507.19912*, Aug. 2025.
- [14] J. Redmon et al., "You only look once: Unified real-time object detection," in *Proc. IEEE CVPR, Las Vegas, 2016*, pp. 779–788.
- [15] F. Yu et al., "BDD100K: A diverse driving dataset," in *Proc. IEEE/CVF CVPR, 2020*, pp. 2636–2645.
- [16] S. S. Mousavi et al., "Traffic light control using deep policy-gradient RL," *IET Intell. Transp. Syst.*, vol. 11, no. 7, pp. 417–423, 2017.
- [17] M. R. Rahman and M. M. Mamun, "Smart traffic signal optimization: A review," *Preprints.org*, 2025.
- [18] S. Srivastava, "Adaptive traffic light timer control (ATLTC)," *Tech. Rep.*, IIT Kanpur–NERD, 2016.
- [19] V. K. Prasad et al., "Chronicles of assaults at cloud computing and its influence at an IaaS," in *Proc. ICIoTCT 2018*, pp. 614–
- [20] T. Eom and B. Kim, "The traffic signal control problem for intersections: A review" *Eur. Transp. Res. Rev.*, vol. 12, art. 20, 2020.

NAMRATA PATEL

UrbanEye : Vision-Based Automated Traffic Optimization Using Computer Vision

 Parul University

Document Details

Submission ID

trn:oid::3117:607538357

Submission Date

Jul 13, 2026, 12:54 PM GMT+5:30

Download Date

Jul 13, 2026, 1:06 PM GMT+5:30

File Name

151_CSE_2.pdf

File Size

1.2 MB

9 Pages

4,779 Words

27,503 Characters

***% detected as AI**

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Vision-Based Automated Traffic Flow Optimization using Computer Vision – Undergraduate Research Paper

UrbanEye : Vision-Based Automated Traffic Optimization Using Computer Vision

Darshan Patel

*Dept. of Computer Engineering
Parul University, Vadodara, India
darshanbpatel228599@gmail.com*

Ogesh Rupareliya

*Dept. of Computer Engineering
Parul University, Vadodara, India
10engogeshrupareliya@gmail.com*

Karmdipsinh Vaghela

*Dept. of Computer Engineering
Parul University, Vadodara, India
karmdipvaghela4@gmail.com*

Hirva Trivedi

*Dept. of Computer Engineering
Parul University, Vadodara, India
hirva1405@gmail.com*

Namrata Patel

*Dept. of Computer Engineering
Parul University, Vadodara, India
namratapatel150894@gmail.com*

Abstract — With so many people living in cities, it's hard to imagine ever having traffic signals that operate on a specific schedule. This paper introduces the Vision-Based Automated Traffic Optimization (VBATO) system. This system is a closed system that uses real-time computer vision, vehicle detection using YOLOv8, traffic lights that adapt based on density, and emergency vehicle priority. The system captures live CCTV footage, detects vehicles by lane, predicts the density by class weight, calculates green light time based on real lane traffic, and activates preemption of signals the moment emergency vehicles are detected. I read twenty peer-reviewed articles that talked about YOLO, deep reinforcement learning, smart cities, and adaptive signal control. I found three holes in research: separating detectors from signal controllers, testing reinforcement learning only in simulation, and testing how well it can work in the worst conditions. The VBATO architecture fills all three gaps by integrating a tightly coupled detection and action system on existing CCTVs.

Keywords — Computer Vision, YOLOv8, Smart Traffic Management, Vehicle Detection, Adaptive Traffic Control, Smart Cities, Deep Learning, Traffic Signal Optimization.

I. INTRODUCTION

Traffic in the cities is one of the most important problems of the twenty-first century. Commuters in large cities waste hundreds of hours a year just because of signal interruptions. Idling cars pollute the air much more than other vehicles. The structural cause is obvious: almost all signalized intersections around the world run on pre-programmed times based on old vehicle counts that cannot react to the changing flow of real traffic.

Two factors made real-time vision-based traffic management almost possible: widespread networked

2025 with emphasis in 2020–2025. Key word: traffic signal optimization, YOLO vehicle detection, adaptive traffic control, deep reinforcement learning (RL) in traffic, and smart transportation system.

B. Traditional and Sensor-Based Systems

The first traffic systems used signal cycles that were set in advance based on traffic data. These systems did not adjust to changes in traffic in time. Traffic systems with sensors like loops worked better but cost a lot. Between 15,000 and 50,000 dollars, for each intersection. This high cost made it hard to use

NAMRATA PATEL

**UrbanEye : Vision-Based Automated Traffic Optimization
Using Computer Vision**

 Parul University

Document Details

Submission ID

trn:oid::3117:607538357

Submission Date

Jul 13, 2026, 12:54 PM GMT+5:30

Download Date

Jul 13, 2026, 1:06 PM GMT+5:30

File Name

151_CSE_2.pdf

File Size

1.2 MB

9 Pages

4,779 Words

27,503 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Abstract
- Methods and Materials
- Preprints

Match Groups

-  **25 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4%  Internet sources
- 5%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Hidden Text**
147 suspect characters on 3 pages
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Chapter 10

References

1. K. Kollias, “Weighted Stackelberg Algorithms for Road Traffic Optimization,” *ACM*, 2021.
2. S. Singh, “Innovative Traffic Optimization Techniques: Integrating Braess’ Paradox and Vision Transformer Architectures (ViT) for Novel Roadway Modalities,” *ACM*, 2023.
3. Z. Wang, “Research on Optimized Control of Urban Traffic Signal Lights Based on PLC and Intelligent Algorithm,” *ACM*, 2025.
4. P. Azevedo, “Comparative Analysis of Multiple YOLO-based Target Detectors and Trackers for ADAS in Edge Devices,” *Elsevier*, 2024.
5. S. Shen, “Traffic Strategy of Signalized Intersection Integrating Vehicle Lane Change and Velocity Trajectory Optimization in Mixed Traffic Environment,” *Elsevier*, 2026.
6. V. Harantová, “The Traffic Lights Timer Optimization Model for a Frequented Intersection in a Small Urban Area in Slovakia,” *Elsevier*, 2021.
7. X. Liang, “Deep Reinforcement Learning for Traffic Light Control in Vehicular Networks,” *IEEE*, 2019.
8. O. A. Ayegbusi, “A Deep Learning-based Model for Traffic Signal Control using the YOLO Algorithm,” *IEEE*, 2025.
9. Y. Quan, “Dynamic Optimization Project Study between the Traffic Organization and the Traffic Signal Control of Urban Traffic,” *IEEE*, 2020.
10. J. Withanawasam, “Multi-agent Based Road Traffic Control Optimization,” *IEEE*, 2017.
11. Lakshay, “AI-Driven Traffic Signal Optimization for Better Congestion Control,” *IEEE*, 2023.
12. T. Tahmid, “Density Based Smart Traffic Control System Using Canny Edge Detection Algorithm for Congregating Traffic Information,” *IEEE*, 2017.

13. B. Manandhar, “Adaptive Traffic Light Control with Statistical Multiplexing Technique and Particle Swarm Optimization in Smart Cities,” *IEEE*, 2018.
14. A. R. Vyas, “Real-Time Traffic Surveillance and Vehicle Speed Detection Using Machine Vision,” *ResearchGate*, 2025.
15. A. Sakhuja, “Intelligent Traffic Management System using Computer Vision and Machine Learning,” *ResearchGate*, 2023.
16. A. A. Ouallane, “Overview of Road Traffic Management Solutions Based on IoT and AI,” *ScienceDirect*, 2021.
17. X. Li, “Intelligent Traffic Management with Mamba-ATSP: Optimizing Signal Control and Path Planning using Multi-modal Data Fusion,” *ScienceDirect*, 2024.
18. A. R. Singh, “Real-time Traffic Flow Optimization using Large Language Models and Reinforcement Learning for Smart Urban Mobility,” *ScienceDirect*, 2024.
19. H. Song, “Vision-based Vehicle Detection and Counting System using Deep Learning in Highway Scenes,” *Springer*, 2019.
20. M. Eom and B.-I. Kim, “The Traffic Signal Control Problem for Intersections: A Review,” *Springer*, 2020.